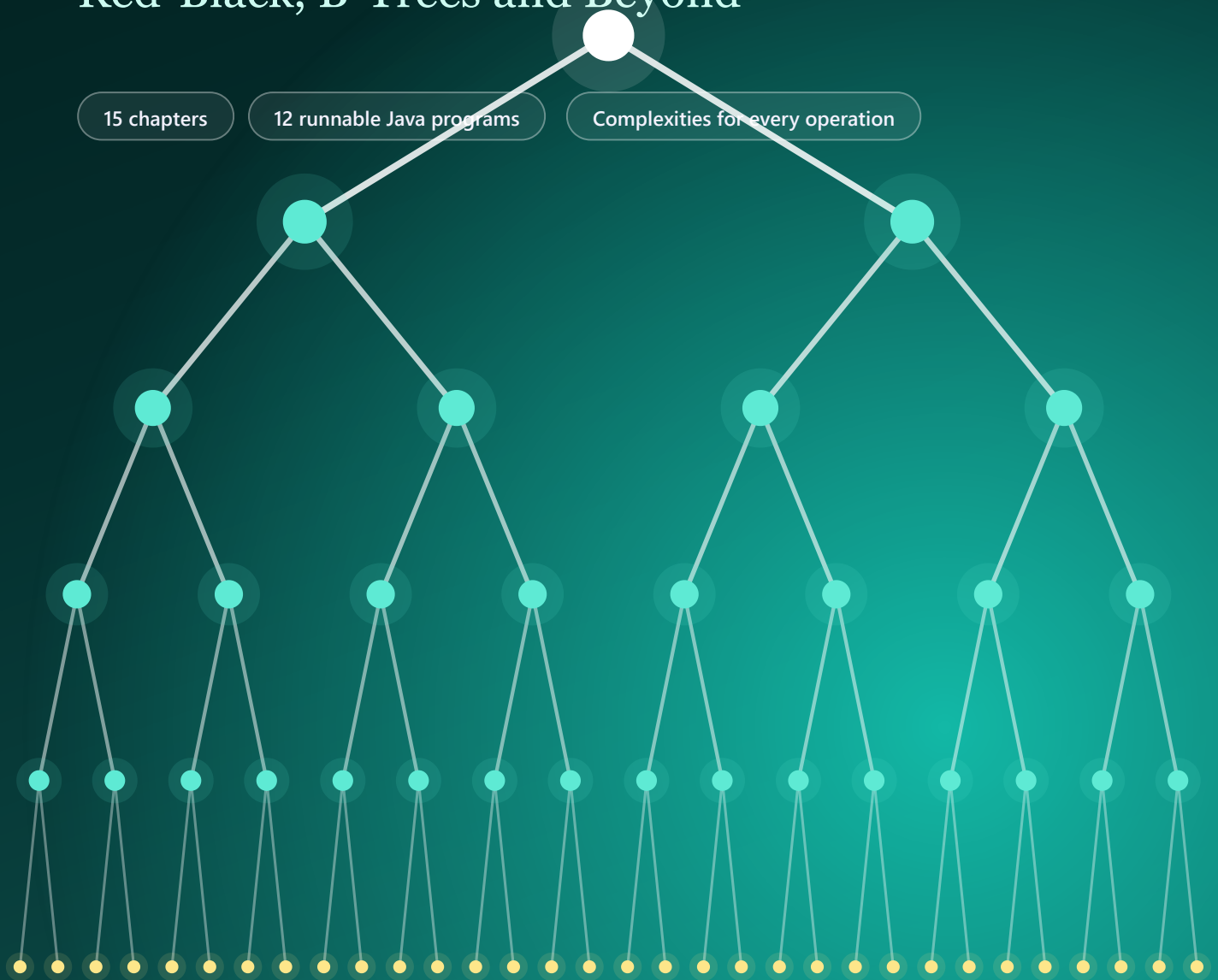


DATA STRUCTURES · JAVA EDITION

Trees

From Binary Trees to AVL,
Red-Black, B-Trees and Beyond



Traversals · BST · AVL · Red-Black · Splay · B-Tree · B+ Tree · Heap · Trie · Segment Tree · Fenwick · Union-Find · Huffman

Contents

PART I	Binary Tree Fundamentals	4
01	What Is a Tree?	5
02	Build a Binary Tree, Part 1: Create, Insert, Print	8
03	Build a Binary Tree, Part 2: Traversals	15
04	Build a Binary Tree, Part 3: Measure, Search, Delete	24
PART II	Search Trees	36
05	Build a Binary Search Tree	37
06	Build an AVL Tree	52
07	Red-Black Trees	63
08	Splay Trees, Treaps and Other Balanced Trees	68
09	B-Trees and B+ Trees	72
PART III	Special-Purpose Trees	77
10	Build a Heap (Priority Queue)	78
11	Tries (Prefix Trees)	86
12	Segment Trees and Fenwick Trees	90
13	N-ary Trees and Tree Applications	94
14	Trees in the Java Collections Framework	102
15	Cheat Sheet: Choosing the Right Tree	105

How to Use This Book

Trees are the most important non-linear data structure in computer science. File systems, databases, compilers, network routers, search engines, autocomplete, priority schedulers, game AI and even the `TreeMap` you use in Java are all trees underneath. This book covers every tree you need to know, from the definition of a node to AVL rotations, red-black trees and B-trees, in a compact, information-dense format.

What you get:

- **Every major tree type:** binary trees, binary search trees, AVL, red-black, splay, treap, B-tree, B+ tree, heaps, tries, segment trees, Fenwick trees, N-ary trees, expression trees, Huffman trees and union-find forests.
- **Code-along chapters.** The four most important structures (**binary tree, binary search tree, AVL tree, heap**) are built **with you, one method at a time**: start from an empty class, add insert, print, traversals, search, height, delete and so on. Each step has a “try it first” exercise and a checkpoint showing the exact output your code should produce.
- **Working Java for everything.** All 12 programs in this book were compiled and run while the book was built; the green *Output* and *Checkpoint* boxes show their real output. The source files are in the `trees-java` folder next to this PDF.
- **Complexities for every operation**, with the reasoning behind them, and a complete cheat sheet in the final chapter.
- **Diagrams** for the tricky parts: rotations, splits, traversal orders.

Conventions used throughout:

Convention	Meaning
n	number of nodes (or keys) in the tree
h	height of the tree
height	counted in edges : a single node has height 0, the empty tree has height -1
depth	number of edges from the root to a node (the root has depth 0)
shape output	a compact picture: <code>50(30, 70)</code> means 50 with children 30 and 70; <code>60(C, 65)</code> means 60 with only a right child

✓ PREREQUISITE

You should be comfortable with Java classes, references, and basic recursion. Almost every tree algorithm is recursive: *solve the problem for the left subtree, solve it for the right subtree, combine the answers at the node*. If recursion feels shaky, keep the companion book *Recursion in Java* nearby.

P A R T

I

Binary Tree Fundamentals

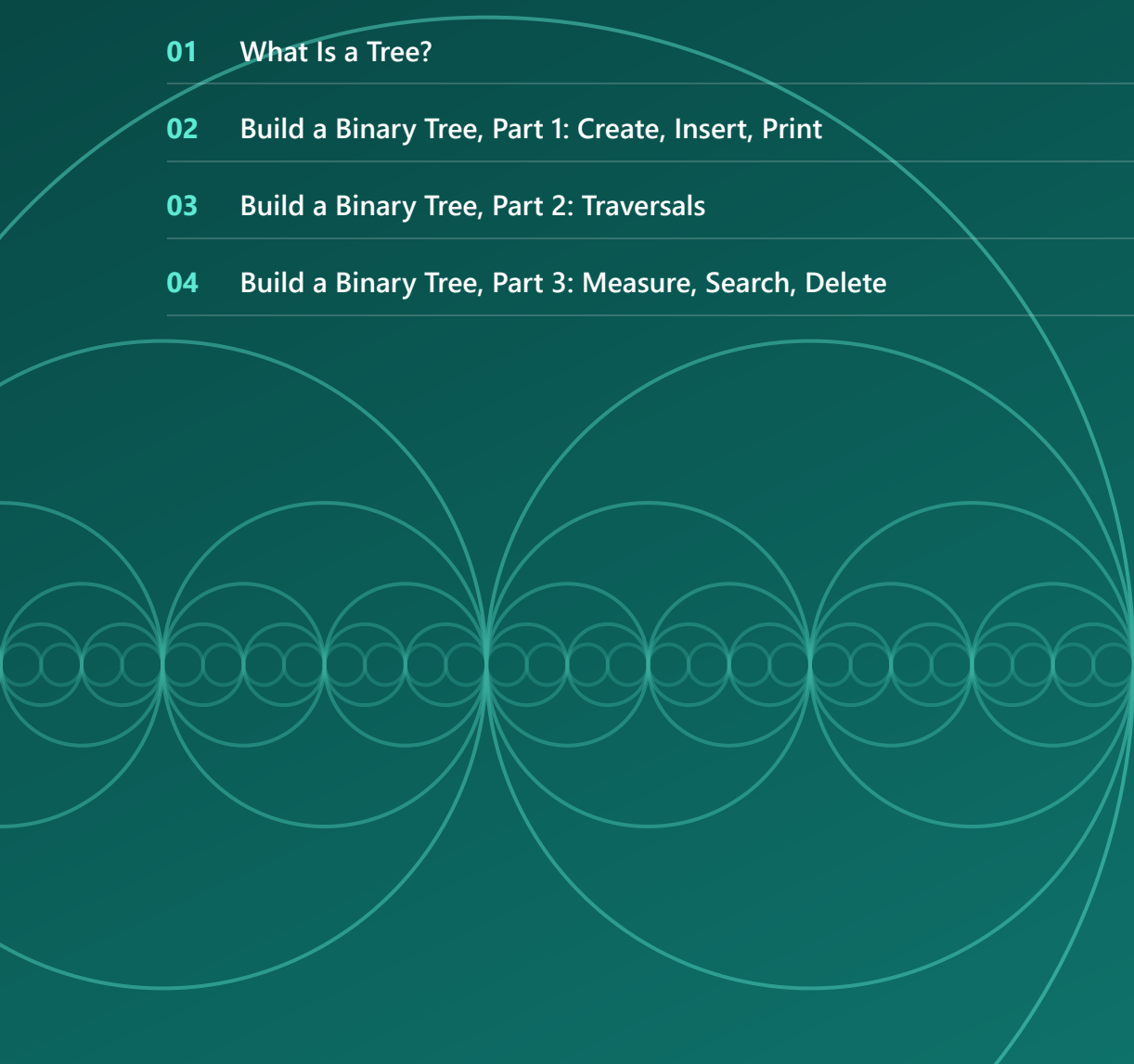
Vocabulary first, then we build a complete BinaryTree class together, one method at a time: creating, inserting, printing, traversing, measuring, deleting.

01 What Is a Tree?

02 Build a Binary Tree, Part 1: Create, Insert, Print

03 Build a Binary Tree, Part 2: Traversals

04 Build a Binary Tree, Part 3: Measure, Search, Delete



CHAPTER 1

What Is a Tree?

01

1.1 From lists to trees

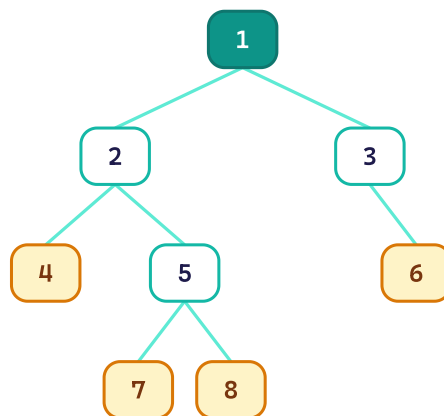
Arrays and linked lists are **linear**: every element has at most one successor. Many real structures are not linear. A folder contains many folders; a company has a CEO with several managers, each with their own team; an HTML page has nested elements. To model **hierarchy**, we need a structure where one element can have several successors. That structure is a **tree**.

★ DEFINITION

A **tree** is a collection of **nodes** connected by **edges**, such that there is exactly one special node called the **root**, every other node has exactly **one parent**, and there are **no cycles**. Equivalently: a tree is either empty, or a root node together with zero or more **subtrees**, each of which is itself a tree.

The second, recursive form of the definition is the one that matters for code: **every child of a node is the root of its own smaller tree**. That is why almost every tree algorithm is recursive.

1.2 Vocabulary



The sample tree used throughout Part I

■ root ■ leaf

Using the tree above:

Term	Meaning	Example
Root	the top node; the only node with no parent	1
Parent / child	a node directly above / below another, joined by an edge	2 is the parent of 4 and 5
Siblings	children of the same parent	4 and 5; 7 and 8
Leaf (external node)	a node with no children	4, 7, 8, 6
Internal node	a node with at least one child	1, 2, 3, 5
Edge	a link between parent and child	1–2, 2–5 ... (7 edges in total)
Path	a sequence of nodes connected by edges	1 → 2 → 5 → 7
Ancestor / descendant	any node on the path up to the root / down from a node	ancestors of 7: 5, 2, 1
Subtree	a node together with all its descendants	the subtree at 5 is
Degree	number of children of a node	degree(5) = 2, degree(3) = 1
Depth of a node	edges from the root to that node	depth(7) = 3
Level	all nodes at the same depth	level 2 =
Height of a node	edges on the longest path from it down to a leaf	height(2) = 2, height(leaf) = 0
Height of the tree	height of the root	3
Forest	a collection of disjoint trees	remove the root → a forest of 2 trees

I DEPTH VS. HEIGHT

Depth is measured from the **top** (root) down to the node. **Height** is measured from the node down to the **bottom** (deepest leaf). The height of the tree equals the maximum depth of any node. Some books count nodes instead of edges (so a single node has height 1); this book always counts **edges**.

1.3 Three facts worth memorizing

- 1. A tree with n nodes has exactly $n - 1$ edges.** Every node except the root has exactly one edge to its parent.
- 2. There is exactly one path between any two nodes.** A second path would form a cycle.
- 3. Every node is the root of a subtree.** Solve a problem for the children's subtrees, combine at the node: that is the template for most tree algorithms.

1.4 Where trees appear

Application	Tree used
File systems, org charts, XML/HTML DOM	general (N-ary) trees
<code>TreeMap</code> , <code>TreeSet</code> , in-memory indexes	red-black trees (balanced BSTs)
Database and file-system indexes	B-trees and B+ trees
Priority queues, schedulers, Dijkstra, heap sort	binary heaps
Autocomplete, spell check, IP routing	tries
Compilers (parse trees, syntax trees)	expression / syntax trees
Compression (ZIP, JPEG, MP3)	Huffman trees
Range queries in competitive programming	segment trees, Fenwick trees
Network connectivity, Kruskal's algorithm	union-find forests

Σ CHAPTER SUMMARY

- A tree is a root plus subtrees: **one parent per node, no cycles**, n nodes and $n - 1$ edges.
- **Depth** counts edges down from the root; **height** counts edges down to the deepest leaf.
- Every node is the root of a subtree, which makes trees naturally recursive.

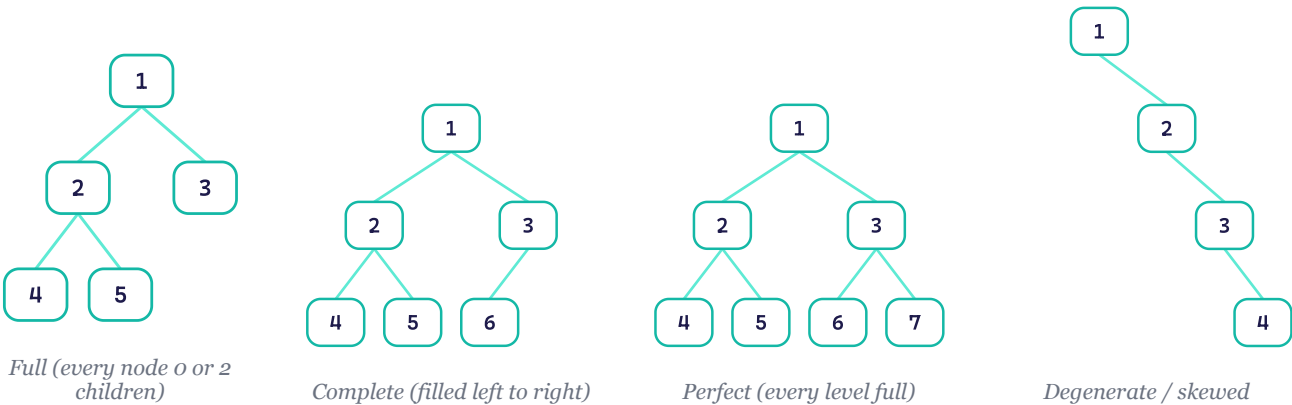
CHAPTER 2

Build a Binary Tree, Part 1: Create, Insert, Print

02

2.1 Binary trees in one page

A **binary tree** is a tree in which every node has **at most two** children, called the **left child** and the **right child**. Left and right are distinct positions: a node with only a left child is a different tree from a node with only a right child.



Type	Definition	Why it matters
Full	every node has 0 or 2 children	Huffman and expression trees are full
Complete	every level full except possibly the last, filled left to right	fits in an array with no gaps: the basis of heaps
Perfect	all internal nodes have 2 children and all leaves are at the same depth	exactly $2^{(h+1)} - 1$ nodes
Balanced	height is $O(\log n)$ (e.g. subtree heights differ by ≤ 1 everywhere)	fast operations: AVL, red-black
Degenerate	every node has at most one child	a linked list in disguise: height $n - 1$

Property (height h , n nodes)	Value
max nodes on level i	2^i
max nodes in the tree	$2^{(h+1)} - 1$
min possible height	$\lceil \log_2 n \rceil$
max possible height	$n - 1$
<code>null</code> child references	$n + 1$

★ WHY HEIGHT IS EVERYTHING

Most tree operations walk from the root down one path, so they cost $O(h)$. A balanced tree with a million nodes has $h \approx 20$; a degenerate one has $h = 999,999$. Keeping h small is the entire point of AVL trees, red-black trees and B-trees.

Array representation. A *complete* tree can live in an array with no pointers: root at index 0, children of index i at $2i + 1$ and $2i + 2$, parent at $(i - 1) / 2$. Heaps use this (Chapter 10). General trees use **linked nodes**, which is what we build now.

2.2 How this code-along works

Over the next three chapters we will write **one class**, `BinaryTree.java`, from an empty file to about 30 methods. Create the file now and follow along. Each step has the same structure:

1. **Goal:** what the method must do (often with a picture).
2. **Try it first:** an orange box. Stop reading and write the method yourself; it's fine if it's wrong.
3. **Code:** the book's version, then an explanation of every important line.
4. **Checkpoint:** code to put in `main`, and the **real output** it produced when this book was built. If your output matches, move on.

✓ THE PUBLIC/PRIVATE PATTERN USED EVERYWHERE

Users of the class call simple methods like `tree.size()`. Inside, a **private recursive helper** does the work on a `Node`: `size()` calls `size(root)`. The public method hides the nodes; the private one follows the recursive definition of a tree. You'll see this pair in almost every step.

2.3 Step 1: The skeleton (Node, root, isEmpty, clear)

Goal: a class that can represent an empty tree.

A tree is made of nodes. Each node stores a value and two references, `left` and `right`, where `null` means "no child on this side". The tree object itself needs only **one field: a reference to the root**. An empty tree is simply `root == null`.

```

BinaryTree.java  skeleton

private static class Node {
    int val;
    Node left, right;           // null = "no child on this side"

    Node(int val) {
        this.val = val;
    }
}

private Node root;             // entry point to the whole tree; null = empty tree

public boolean isEmpty() {
    return root == null;
}

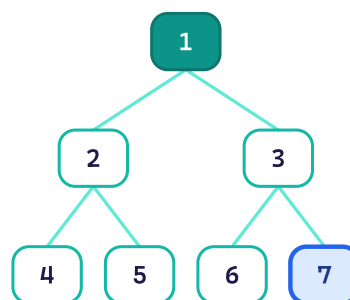
public void clear() {
    root = null;                // the garbage collector frees all the nodes
}

```

- `Node` is a `private static` nested class: users of `BinaryTree` never see nodes, only values. `static` means a node doesn't need a reference to the tree object.
- `clear()` just drops the root. Every node becomes unreachable, and Java's garbage collector frees them. There's no need to visit each node, unlike in C or C++.

2.4 Step 2: insert(value)

Goal: add a value to the tree. A plain binary tree has no ordering rule, so *where* should a new node go? The most useful choice is **the first free position in level order** (top to bottom, left to right). That keeps the tree **complete**, and therefore as short as possible.



insert(7): the first free slot in level order

■ root ■ new node

? TRY IT FIRST

Hint: visiting nodes in level order needs a **queue**. Visit nodes one by one; the first node that is missing a left or right child is where the new node goes.

BinaryTree.java insert

```
// Adds the value at the first free position in level order (top to bottom,
// left to right), so the tree stays complete.
public void insert(int value) {
    Node fresh = new Node(value);
    if (root == null) {          // empty tree: the new node becomes the root
        root = fresh;
        return;
    }
    Queue<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (true) {
        Node cur = queue.poll();
        if (cur.left == null) {   // first hole found: fill it
            cur.left = fresh;
            return;
        }
        queue.add(cur.left);
        if (cur.right == null) {
            cur.right = fresh;
            return;
        }
        queue.add(cur.right);
    }
}
```

- If the tree is empty, the new node becomes the root: that's the base case of building a tree.
- Otherwise we do a **breadth-first search**: take a node off the queue; if its left slot is empty, fill it and stop; otherwise enqueue the left child and check the right slot the same way.
- The `while (true)` loop always ends, because a finite tree always has an empty slot somewhere.

Complexity: $O(n)$ time, because in the worst case we visit every node before finding the hole, and $O(n)$ memory for the queue. (Binary search trees, in Chapter 5, find the spot in $O(h)$ by using an ordering rule.)

2.5 Step 3: print() and shape()

Goal: see the tree. We want two views: a drawing, like the `tree` command in a terminal, and a compact one-line form for quick comparisons.

? TRY IT FIRST

Write `shape()` first: for a node, print its value, then `(left,right)` recursively; for a leaf just print the value. The drawing is harder: each line needs a *prefix* of `|` and spaces that depends on the ancestors.

BinaryTree.java print

```
// Draws the tree like the "tree" command, labelling each child L or R.
public void print() {
```

```

    StringBuilder sb = new StringBuilder();
    if (root == null) {
        sb.append("(empty tree)\n");
    } else {
        sb.append(root.val).append('\n');
        printChildren(root, "", sb);
    }
    System.out.print(sb);
}

private void printChildren(Node node, String prefix, StringBuilder sb) {
    List<Node> kids = new ArrayList<>();
    List<String> sides = new ArrayList<>();
    if (node.left != null) { kids.add(node.left); sides.add("L"); }
    if (node.right != null) { kids.add(node.right); sides.add("R"); }
    for (int i = 0; i < kids.size(); i++) {
        boolean last = i == kids.size() - 1;
        sb.append(prefix).append(last ? "└─ " : "│─ ")
          .append(sides.get(i)).append(": ").append(kids.get(i).val).append('\n');
        printChildren(kids.get(i), prefix + (last ? "    " : "│   "), sb);
    }
}

// One-line picture: val(left,right); "3(,6)" means 3 has only a right child.
public String shape() {
    return shape(root);
}

private String shape(Node n) {
    if (n == null) return "";
    if (n.left == null && n.right == null) return String.valueOf(n.val);
    return n.val + "(" + shape(n.left) + "," + shape(n.right) + ")";
}

```

- `printChildren` is a **preorder** traversal (node, then children). It carries a `prefix` string down the recursion. When a child is the last one printed at its level, its descendants get `" "` (blank) instead of `"|"`, because no more siblings follow below.
- Each child is labeled `L` or `R`. A tree printer that ignores sides can't show the difference between a left-only and a right-only child.
- `shape()` shows the same structure in one line: `3(,6)` means 3 has no left child and a right child 6.

Checkpoint 1. Put this in `main`, run it, and compare:

```

BinaryTree.java  checkCreate

BinaryTree tree = new BinaryTree();
System.out.println("empty? " + tree.isEmpty());
for (int v = 1; v <= 6; v++) tree.insert(v);
System.out.println("empty? " + tree.isEmpty() + "    shape: " + tree.shape());
tree.print();

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, CREATE (ACTUAL RUN)

```

empty? true
empty? false   shape: 1(2(4,5),3(6,))
1
├─ L: 2
│  ├── L: 4
│  └─ R: 5
└─ R: 3
    └─ L: 6

```

Insert filled positions 1 to 6 in level order: 2 and 3 under 1, then 4, 5 under 2, then 6 as the left child of 3.

2.6 Step 4: fromLevelOrder(values...), building any shape

Goal: `insert` always produces a complete tree. To test algorithms on irregular trees we need to build **any** shape. The standard trick (used by LeetCode) is a level-order list where `null` marks a missing node.

```

BinaryTree.java  fromLevelOrder

// Builds any shape: values in level order, null = "no node at this position".
public static BinaryTree fromLevelOrder(Integer... values) {
    BinaryTree tree = new BinaryTree();
    if (values.length == 0 || values[0] == null) return tree;
    tree.root = new Node(values[0]);
    Queue<Node> queue = new ArrayDeque<>();
    queue.add(tree.root);
    int i = 1;
    while (!queue.isEmpty() && i < values.length) {
        Node cur = queue.poll();
        if (i < values.length && values[i] != null) {    // left child of cur
            cur.left = new Node(values[i]);
            queue.add(cur.left);
        }
        i++;
        if (i < values.length && values[i] != null) {    // right child of cur
            cur.right = new Node(values[i]);
            queue.add(cur.right);
        }
        i++;
    }
    return tree;
}

```

- It is a `static` **factory method**: it creates and returns a new tree. Being inside the class, it can set the private `root` field directly.
- The queue holds nodes that are still waiting for their children. Each node consumes the **next two values** in the array: left, then right. A `null` value consumes a position but creates no node.
- `Integer...` (varargs) lets us write `fromLevelOrder(1, 2, 3, null, 5)`. We need `Integer`, not `int`, because `int` can't be `null`.

Checkpoint 2. Build the sample tree from Chapter 1. We'll use it (as `t`) in all the following steps.

BinaryTree.java checkSample

```
BinaryTree t = BinaryTree.fromLevelOrder(1, 2, 3, 4, 5, null, 6, null, null, 7, 8);
t.print();
System.out.println("shape: " + t.shape());
```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, SAMPLE (ACTUAL RUN)

```
1
├── L: 2
│   ├── L: 4
│   └── R: 5
│       ├── L: 7
│       └── R: 8
└── R: 3
    └── R: 6
shape: 1(2(4,5(7,8)),3(,6))
```

Σ CHAPTER SUMMARY: OUR CLASS SO FAR

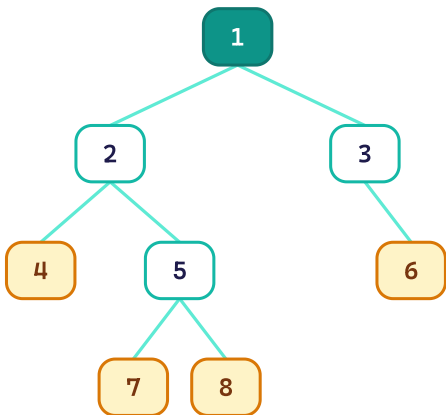
- `Node` + `root` is all the state a binary tree needs; `root == null` means empty.
- `insert` fills the first free level-order slot using a **queue**: $O(n)$, keeps the tree complete.
- `print` / `shape` let you *see* every later step; `fromLevelOrder` builds any test shape.
- Public methods call private recursive helpers on nodes.

CHAPTER 3

Build a Binary Tree, Part 2: Traversals

03

A **traversal** visits every node exactly once. Trees have several natural orders, and choosing the right one is often the key to solving a problem. We add them to `BinaryTree` in five steps. Keep the sample tree in mind:



The sample tree used throughout Part I

root leaf

3.1 Step 5: preorder, inorder, postorder (recursive)

Goal: return the values in each of the three **depth-first** orders. They differ only in **when the node itself is visited** relative to its subtrees:

Traversal	Order	Sample tree	Typical uses
Preorder	Node, Left, Right	1 2 4 5 7 8 3 6	copy/serialize a tree, print a hierarchy
Inorder	Left, Node, Right	4 2 7 5 8 1 3 6	sorted order in a BST
Postorder	Left, Right, Node	4 7 8 5 2 6 3 1	delete a tree, compute sizes, evaluate expressions

? TRY IT FIRST

Each one is 4 lines: `if (n == null) return;`, then the three actions in the right order. Use a public method that creates a list and a private helper that fills it.

```
BinaryTree.java dfsRecursive

public List<Integer> preorder() {
    List<Integer> out = new ArrayList<>();
    preorder(root, out);
    return out;
}
```

```

}

private void preorder(Node n, List<Integer> out) {    // Node, Left, Right
    if (n == null) return;
    out.add(n.val);
    preorder(n.left, out);
    preorder(n.right, out);
}

public List<Integer> inorder() {
    List<Integer> out = new ArrayList<>();
    inorder(root, out);
    return out;
}

private void inorder(Node n, List<Integer> out) {    // Left, Node, Right
    if (n == null) return;
    inorder(n.left, out);
    out.add(n.val);
    inorder(n.right, out);
}

public List<Integer> postorder() {
    List<Integer> out = new ArrayList<>();
    postorder(root, out);
    return out;
}

private void postorder(Node n, List<Integer> out) {    // Left, Right, Node
    if (n == null) return;
    postorder(n.left, out);
    postorder(n.right, out);
    out.add(n.val);
}

```

The helper adds to **one shared list** that is passed down, instead of creating and concatenating lists at every node, which would cost $O(n^2)$. Each node is visited once: **$O(n)$ time, $O(h)$ stack space.**

Checkpoint 3.

BinaryTree.java checkDfs

```

System.out.println("preorder : " + t.preorder());
System.out.println("inorder : " + t.inorder());
System.out.println("postorder: " + t.postorder());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, DFS (ACTUAL RUN)

```

preorder : [1, 2, 4, 5, 7, 8, 3, 6]
inorder : [4, 2, 7, 5, 8, 1, 3, 6]
postorder: [4, 7, 8, 5, 2, 6, 3, 1]

```

✦ PRODUCE THE ORDERS BY HAND

Trace a line around the tree counter-clockwise from the root, hugging every node. Record a node when you pass its **left** side for preorder, when you pass **underneath** it for inorder, and when you pass its **right** side for postorder.

3.2 Step 6: the same traversals with a stack

Goal: the same three orders **without recursion**. This matters for very deep trees (recursion could overflow Java's stack), for iterators that must pause between elements, and in interviews.

? TRY IT FIRST

Preorder: push the root; loop: pop, record, push right then left. Why right first?

```
BinaryTree.java  dfsIterative

public List<Integer> preorderIterative() {
    List<Integer> out = new ArrayList<>();
    Deque<Node> stack = new ArrayDeque<>();
    if (root != null) stack.push(root);
    while (!stack.isEmpty()) {
        Node n = stack.pop();
        out.add(n.val);
        if (n.right != null) stack.push(n.right); // pushed first, popped later
        if (n.left != null) stack.push(n.left);   // pushed last, popped next
    }
    return out;
}

public List<Integer> inorderIterative() {
    List<Integer> out = new ArrayList<>();
    Deque<Node> stack = new ArrayDeque<>();
    Node cur = root;
    while (cur != null || !stack.isEmpty()) {
        while (cur != null) { // slide down the left spine
            stack.push(cur);
            cur = cur.left;
        }
        cur = stack.pop(); // leftmost unvisited node
        out.add(cur.val);
        cur = cur.right;   // continue in its right subtree
    }
    return out;
}

public List<Integer> postorderIterative() {
    List<Integer> out = new ArrayList<>();
    Deque<Node> stack = new ArrayDeque<>();
    Node cur = root, lastVisited = null;
}
```

```

while (cur != null || !stack.isEmpty()) {
    if (cur != null) {
        stack.push(cur);
        cur = cur.left;
    } else {
        Node top = stack.peek();
        if (top.right != null && top.right != lastVisited) {
            cur = top.right;           // right subtree not done yet
        } else {
            out.add(top.val);           // both subtrees done
            lastVisited = stack.pop();
        }
    }
}
return out;
}

```

- **Preorder**: pop a node, record it, push its children. The **right child is pushed first** so the left child is on top and comes out first.
- **Inorder**: slide down the left spine pushing every node; the popped node is the leftmost unvisited one; record it, then continue from its right child. `TreeMap`'s iterator works exactly like this.
- **Postorder**: a node may only be recorded after **both** subtrees. `lastVisited` tells us whether we're coming back up from the right subtree (record now) or should go down into it first.

Checkpoint 4. The results must equal checkpoint 3.

BinaryTree.java checkIterative

```

System.out.println("preorder (stack): " + t.preorderIterative());
System.out.println("inorder (stack): " + t.inorderIterative());
System.out.println("postorder (stack): " + t.postorderIterative());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, ITERATIVE (ACTUAL RUN)

```

preorder (stack): [1, 2, 4, 5, 7, 8, 3, 6]
inorder (stack): [4, 2, 7, 5, 8, 1, 3, 6]
postorder (stack): [4, 7, 8, 5, 2, 6, 3, 1]

```

3.3 Step 7: level order (breadth-first)

Goal: return the nodes **level by level**, as a list of lists.

```

BinaryTree.java levelOrder

public List<List<Integer>> levelOrder() {
    List<List<Integer>> levels = new ArrayList<>();
    Queue<Node> queue = new ArrayDeque<>();
    if (root != null) queue.add(root);
    while (!queue.isEmpty()) {
        int count = queue.size();           // nodes on the current level
        List<Integer> level = new ArrayList<>();
        for (int k = 0; k < count; k++) {
            Node n = queue.poll();
            level.add(n.val);
            if (n.left != null) queue.add(n.left);
            if (n.right != null) queue.add(n.right);
        }
        levels.add(level);
    }
    return levels;
}

```

A **queue** gives first-in, first-out order: children are processed after everything already waiting, which is exactly “level by level”. The trick is `int count = queue.size()` at the start of each round: that is exactly the number of nodes on the current level, so the inner loop processes one level. **$O(n)$ time, $O(w)$ memory**, where w is the widest level.

3.4 Step 8: zigzag and the four views

Goal: common interview variations, built on top of `levelOrder`.

```

BinaryTree.java variations

public List<List<Integer>> zigzag() {
    List<List<Integer>> result = new ArrayList<>();
    boolean leftToRight = true;
    for (List<Integer> level : levelOrder()) {
        LinkedList<Integer> row = new LinkedList<>();
        for (int v : level) {
            if (leftToRight) row.addLast(v);
            else row.addFirst(v);           // reverse every other level
        }
        result.add(row);
        leftToRight = !leftToRight;
    }
    return result;
}

public List<Integer> leftView() {           // first node of every level
    List<Integer> view = new ArrayList<>();
    for (List<Integer> level : levelOrder()) view.add(level.get(0));
    return view;
}

```

```

public List<Integer> rightView() {                                // last node of every level
    List<Integer> view = new ArrayList<>();
    for (List<Integer> level : levelOrder()) view.add(level.get(level.size() - 1));
    return view;
}

private record Item(Node node, int col) {}                      // col: root 0, left -1, right +1

public List<Integer> topView() {
    TreeMap<Integer, Integer> firstInColumn = new TreeMap<>();
    Queue<Item> queue = new ArrayDeque<>();
    if (root != null) queue.add(new Item(root, 0));
    while (!queue.isEmpty()) {
        Item it = queue.poll();
        firstInColumn.putIfAbsent(it.col(), it.node().val);    // BFS: first = highest
        if (it.node().left != null) queue.add(new Item(it.node().left, it.col() - 1));
        if (it.node().right != null) queue.add(new Item(it.node().right, it.col() + 1));
    }
    return new ArrayList<>(firstInColumn.values());
}

```

- **Zigzag** reverses every second level by adding to the **front** of a `LinkedList` instead of the back.
- **Left/right view**: what you'd see standing to the left or right of the tree: the first or last node of each level.
- **Top view**: give every node a column number (root 0, left child -1, right child +1). In BFS order, the **first** node reaching a column is the highest one, so `putIfAbsent` keeps it. A `TreeMap` keeps the columns sorted left to right.

Checkpoint 5.

BinaryTree.java checkLevels

```

System.out.println("level order: " + t.levelOrder());
System.out.println("zigzag      : " + t.zigzag());
System.out.println("left view  : " + t.leftView() + "    right view: " + t.rightView());
System.out.println("top view   : " + t.topView());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, LEVELS (ACTUAL RUN)

```

level order: [[1], [2, 3], [4, 5, 6], [7, 8]]
zigzag      : [[1], [3, 2], [4, 5, 6], [8, 7]]
left view   : [1, 2, 4, 7]    right view: [1, 3, 6, 8]
top view    : [4, 2, 1, 3, 6]

```

3.5 Step 9: Morris inorder, O(1) extra memory

Goal: inorder traversal with **no stack and no recursion**. The trick: temporarily turn the `null` right pointer of each node's **inorder predecessor** into a "thread" pointing back to the node, so we can climb back up later. Then remove it.

```

BinaryTree.java morris

// Inorder with O(1) extra memory: temporary links from each predecessor back up.
public List<Integer> morrisInorder() {
    List<Integer> out = new ArrayList<>();
    Node cur = root;
    while (cur != null) {
        if (cur.left == null) {
            out.add(cur.val);
            cur = cur.right;
        } else {
            Node pred = cur.left;           // rightmost node of left subtree
            while (pred.right != null && pred.right != cur) pred = pred.right;
            if (pred.right == null) {
                pred.right = cur;           // make a thread, go left
                cur = cur.left;
            } else {
                pred.right = null;           // thread used: remove it
                out.add(cur.val);
                cur = cur.right;
            }
        }
    }
    return out;
}

```

For a node with a left subtree: find the predecessor (the rightmost node of the left subtree). If its `right` is `null`, this is the first visit: create the thread and go left. If the thread already exists, we have come back around: the left subtree is finished, so remove the thread, record the node, and go right. Each edge is followed a constant number of times: still $O(n)$, and the tree is fully restored.

3.6 Step 10: rebuild a tree from preorder + inorder

Goal: given the preorder and inorder sequences of a tree with **distinct** values, rebuild the tree. The first preorder value is the root; its position in the inorder sequence splits everything else into the left subtree (values before it) and the right subtree (values after it).

```

BinaryTree.java  rebuild

// Rebuilds a tree from its preorder and inorder sequences (distinct values).
public static BinaryTree fromPreIn(List<Integer> pre, List<Integer> in) {
    Map<Integer, Integer> pos = new HashMap<>();
    for (int i = 0; i < in.size(); i++) pos.put(in.get(i), i);
    BinaryTree tree = new BinaryTree();
    tree.root = build(pre, new int[]{0}, 0, in.size() - 1, pos);
    return tree;
}

private static Node build(List<Integer> pre, int[] next, int lo, int hi,
                           Map<Integer, Integer> pos) {
    if (lo > hi) return null;
    Node n = new Node(pre.get(next[0]++));           // next preorder value = subtree root
    int mid = pos.get(n.val);                         // splits inorder: left | root | right
    n.left = build(pre, next, lo, mid - 1, pos);
    n.right = build(pre, next, mid + 1, hi, pos);
    return n;
}

```

`next` is a one-element array used as a shared counter, so every recursive call consumes preorder values in order. The `HashMap` finds a value's inorder position in $O(1)$, which makes the rebuild **$O(n)$** instead of $O(n^2)$.

Checkpoint 6.

```

BinaryTree.java  checkMorris

System.out.println("morris inorder: " + t.morrisInorder());
System.out.println("tree unchanged: " + t.shape());
BinaryTree copy = BinaryTree.fromPreIn(t.preorder(), t.inorder());
System.out.println("rebuilt from preorder + inorder: " + copy.shape());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, MORRIS (ACTUAL RUN)

```

morris inorder: [4, 2, 7, 5, 8, 1, 3, 6]
tree unchanged: 1(2(4,5(7,8)),3(,6))
rebuilt from preorder + inorder: 1(2(4,5(7,8)),3(,6))

```

Traversal	Time	Extra memory
pre/in/postorder, recursive	$O(n)$	$O(h)$ call stack
pre/in/postorder, iterative	$O(n)$	$O(h)$ explicit stack
level order	$O(n)$	$O(w)$ queue
Morris inorder	$O(n)$	$O(1)$

Σ CHAPTER SUMMARY

- DFS orders: **pre** (N L R), **in** (L N R), **post** (L R N), recursive or with a stack.
- BFS with a queue; `queue.size()` separates levels. Zigzag and views are small variations.
- Morris traversal reaches O(1) memory with temporary threads. Preorder + inorder uniquely rebuild a tree.

CHAPTER 4

Build a Binary Tree, Part 3: Measure, Search, Delete

04

Almost everything in this chapter uses the same recursive shape:

● ● ● The universal tree recursion

```
Result solve(Node node) {
    if (node == null) return BASE;           // answer for the empty tree
    Result left = solve(node.left);          // trust recursion for the left subtree
    Result right = solve(node.right);        // ...and for the right subtree
    return combine(node, left, right);       // build this node's answer
}
```

Only the base value and the combine step change from method to method.

4.1 Step 11: size, height, countLeaves

Goal: the three basic measurements.

? TRY IT FIRST

What should each method return for `null`? Size: 0 nodes. Height: -1 (so a leaf gets $1 + \max(-1, -1) = 0$). Leaves: 0. And what makes a node a leaf?

```

BinaryTree.java  sizeHeight

public int size() {
    return size(root);
}

private int size(Node n) {
    if (n == null) return 0;
    return 1 + size(n.left) + size(n.right);
}

public int height() {
    return height(root);
}

private int height(Node n) {
    if (n == null) return -1;
    return 1 + Math.max(height(n.left), height(n.right));
}

public int countLeaves() {
    return countLeaves(root);
}

private int countLeaves(Node n) {
    if (n == null) return 0;
    if (n.left == null && n.right == null) return 1;
    return countLeaves(n.left) + countLeaves(n.right);
}

```

- `size` = me (1) + size of left + size of right.
- `height` = 1 edge down to my taller child, plus that child's height. Returning **-1** for `null` makes a leaf's height 0, matching our edge-counting convention.
- `countLeaves` needs an **extra base case**: a node with no children counts as 1.

All three: **O(n)** time, **O(h)** stack.

4.2 Step 12: contains, depth, min, max

Goal: search questions. A plain binary tree has no ordering, so we must potentially look **everywhere**.

```

BinaryTree.java  search

public boolean contains(int value) {
    return contains(root, value);
}

private boolean contains(Node n, int value) {
    // no ordering: must look everywhere
    if (n == null) return false;
    return n.val == value || contains(n.left, value) || contains(n.right, value);
}

```

```

public int depth(int value) {                                // edges from root; -1 if absent
    return depth(root, value, 0);
}

private int depth(Node n, int value, int d) {
    if (n == null) return -1;
    if (n.val == value) return d;
    int left = depth(n.left, value, d + 1);
    return left != -1 ? left : depth(n.right, value, d + 1);
}

public int max() {
    if (root == null) throw new IllegalStateException("empty tree");
    return max(root);
}

private int max(Node n) {
    if (n == null) return Integer.MIN_VALUE;                // neutral value for max
    return Math.max(n.val, Math.max(max(n.left), max(n.right)));
}

public int min() {
    if (root == null) throw new IllegalStateException("empty tree");
    return min(root);
}

private int min(Node n) {
    if (n == null) return Integer.MAX_VALUE;                // neutral value for min
    return Math.min(n.val, Math.min(min(n.left), min(n.right)));
}

```

- `contains` uses `||` **short-circuiting**: once one side finds the value, the other side isn't searched.
- `depth` passes the current depth **down** as a parameter (top-down information), unlike `height`, which comes **up** from the children.
- `min` / `max` use neutral values for the empty subtree: `Integer.MAX_VALUE` for min, `Integer.MIN_VALUE` for max. The public methods throw on an empty tree, because it has no minimum.

All **O(n)**. A BST answers these in **O(h)** (Chapter 5).

Checkpoint 7.

BinaryTree.java checkQueries

```

System.out.println("size=" + t.size() + " height=" + t.height() + " leaves=" +
    t.countLeaves());
System.out.println("contains(7)=" + t.contains(7) + " contains(9)=" + t.contains(9));
System.out.println("depth(7)=" + t.depth(7) + " depth(3)=" + t.depth(3)
    + " min=" + t.min() + " max=" + t.max());

```

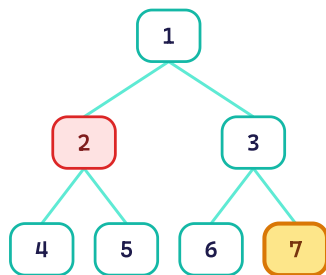
>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, QUERIES (ACTUAL RUN)

```
size=8 height=3 leaves=4
contains(7)=true contains(9)=false
depth(7)=3 depth(3)=1 min=1 max=8
```

4.3 Step 13: delete(value)

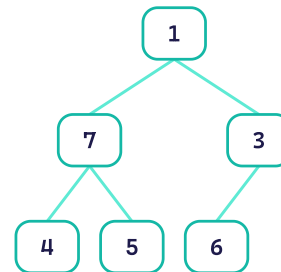
Goal: remove a value while keeping the tree **complete**. We can't just unlink the node: its children would be lost. The standard method for a plain binary tree:

1. Find the node holding the value (the **target**).
2. Find the **deepest, rightmost** node (the last node in level order).
3. Copy its value into the target, then **remove the deepest node**, which is always a leaf.



delete(2): 2 is the target, 7 is the deepest-rightmost node

 node to delete deepest, rightmost node



7's value moves up; the last leaf is unlinked

? TRY IT FIRST

One BFS can find both the target and the last node. The tricky part is removing the last node: you need its **parent**, so remember which node enqueued the most recent child.

```

BinaryTree.java delete

// Replace the value with the deepest, rightmost node's value, then remove that node.
// The tree keeps its shape (a complete tree stays complete).
public boolean delete(int value) {
    if (root == null) return false;
    if (root.left == null && root.right == null) { // single node
        if (root.val != value) return false;
        root = null;
        return true;
    }
    Node target = null, last = null, lastParent = null;
    Queue<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (!queue.isEmpty()) { // one BFS finds both nodes
        Node cur = queue.poll();
        if (target == null && cur.val == value) target = cur;
        if (cur.left != null) { lastParent = cur; queue.add(cur.left); }
        if (cur.right != null) { lastParent = cur; queue.add(cur.right); }
        last = cur; // ends as the deepest, rightmost
    }
    if (target == null) return false;
    target.val = last.val; // copy the value up...
    if (lastParent.right == last) lastParent.right = null; // ...and unlink the node
    else lastParent.left = null;
    return true;
}

```

- A single-node tree is a special case: deleting its value makes the tree empty.
- During the BFS, `last` ends up as the final node dequeued, the deepest and rightmost. `lastParent` is updated every time a child is enqueued, so it ends as the parent of that final node.
- The method returns `false` if the value isn't present: the caller learns whether anything happened.

Complexity: $O(n)$ for the BFS. **Checkpoint 8.**

```

BinaryTree.java checkDelete

BinaryTree d = new BinaryTree();
for (int v = 1; v <= 7; v++) d.insert(v);
System.out.println("start      : " + d.shape());
System.out.println("delete 2 : " + d.delete(2) + " -> " + d.shape());
System.out.println("delete 1 : " + d.delete(1) + " -> " + d.shape());
System.out.println("delete 9 : " + d.delete(9) + " -> " + d.shape());
d.print();

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, DELETE (ACTUAL RUN)

```

start      : 1(2(4,5),3(6,7))
delete 2   : true -> 1(7(4,5),3(6,))
delete 1   : true -> 6(7(4,5),3)
delete 9   : false -> 6(7(4,5),3)
6
├─ L: 7
│   └─ L: 4
│       └─ R: 5
└─ R: 3

```

Deleting 2 copied 7 (the last node) into its place. Deleting the root 1 copied 6, the new last node. Deleting a missing value changes nothing.

4.4 Step 14: isFull, isComplete, isPerfect

Goal: check the tree types from Chapter 2.

```

BinaryTree.java  types

public boolean isFull() {                                // every node has 0 or 2 children
    return isFull(root);
}

private boolean isFull(Node n) {
    if (n == null) return true;
    if ((n.left == null) != (n.right == null)) return false;
    return isFull(n.left) && isFull(n.right);
}

public boolean isComplete() {                            // no node after the first gap
    Queue<Node> queue = new LinkedList<>();              // LinkedList accepts null
    queue.add(root);
    boolean seenGap = false;
    while (!queue.isEmpty()) {
        Node n = queue.poll();
        if (n == null) {
            seenGap = true;
            continue;
        }
        if (seenGap) return false;
        queue.add(n.left);
        queue.add(n.right);
    }
    return true;
}

public boolean isPerfect() {                             // n = 2^(h+1) - 1
    return size() == (1 << (height() + 1)) - 1;
}

```

- `isFull`: a node with exactly one child breaks the rule. `(left == null) != (right == null)` is true exactly when one side is missing.
- `isComplete`: BFS that also enqueues `null` children. Once a `null` appears, no real node may follow. (`LinkedList` is used because `ArrayDeque` refuses `null`.)
- `isPerfect`: a perfect tree of height h has exactly $2^{(h+1)} - 1$ nodes, so compare the size with that formula.

Checkpoint 9.

BinaryTree.java checkTypes

```

BinaryTree perfect = new BinaryTree();
for (int v = 1; v <= 7; v++) perfect.insert(v);
System.out.println("sample : full=" + t.isFull() + " complete=" + t.isComplete()
    + " perfect=" + t.isPerfect());
System.out.println("1..6 : full=" + tree.isFull() + " complete=" + tree.isComplete()
    + " perfect=" + tree.isPerfect());
System.out.println("1..7 : full=" + perfect.isFull() + " complete=" + perfect.isComplete()
    + " perfect=" + perfect.isPerfect());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, TYPES (ACTUAL RUN)

```

sample : full=false complete=false perfect=false
1..6    : full=false complete=true perfect=false
1..7    : full=true complete=true perfect=true

```

4.5 Step 15: isBalanced in O(n)

Goal: check that at every node the heights of the two subtrees differ by at most 1 (the AVL rule). Calling `height` at every node would be $O(n^2)$. Instead, compute heights bottom-up and return a **sentinel** as soon as an imbalance is found:

BinaryTree.java balanced

```

private static final int UNBALANCED = Integer.MIN_VALUE;

public boolean isBalanced() {
    return checkHeight(root) != UNBALANCED;
}

private int checkHeight(Node n) {
    if (n == null) return -1;
    int l = checkHeight(n.left);
    if (l == UNBALANCED) return UNBALANCED;
    int r = checkHeight(n.right);
    if (r == UNBALANCED) return UNBALANCED;
    if (Math.abs(l - r) > 1) return UNBALANCED; // stop as soon as it breaks
    return 1 + Math.max(l, r);
}

```

4.6 Step 16: diameter

Goal: the number of edges on the longest path between **any** two nodes. Every path has a highest node where it bends; the longest path bending at node n has `height(left) + height(right) + 2` edges.

```
BinaryTree.java  diameter

public int diameter() {                                // longest path, in edges
    int[] best = {0};
    heightAndBest(root, best);
    return best[0];
}

private int heightAndBest(Node n, int[] best) {
    if (n == null) return -1;
    int l = heightAndBest(n.left, best);
    int r = heightAndBest(n.right, best);
    best[0] = Math.max(best[0], l + r + 2);             // path that bends at n
    return 1 + Math.max(l, r);                         // what the parent needs
}
```

✦ RETURN WHAT THE PARENT NEEDS, RECORD WHAT THE PROBLEM ASKS

The helper **returns** the height (the parent needs that) and **records** the best diameter in a one-element array (a shared mutable box). One pass, **O(n)**. The same pattern solves maximum path sum in Step 19.

4.7 Step 17: mirror and isSymmetric

```

BinaryTree.java  mirror

public void mirror() {
    mirror(root);
}

private void mirror(Node n) {
    if (n == null) return;
    Node tmp = n.left;
    n.left = n.right;
    n.right = tmp;
    mirror(n.left);
    mirror(n.right);
}

public boolean isSymmetric() {
    return root == null || isMirror(root.left, root.right);
}

private boolean isMirror(Node a, Node b) {
    if (a == null || b == null) return a == b;
    return a.val == b.val && isMirror(a.left, b.right) && isMirror(a.right, b.left);
}

```

`mirror` swaps the children of every node. `isSymmetric` compares the left subtree with the mirror image of the right: outer with outer, inner with inner. `if (a == null || b == null) return a == b;` handles all null cases in one line.

4.8 Step 18: lowest common ancestor

Goal: the deepest node that has both p and q as descendants.

```

BinaryTree.java  lca

public int lca(int p, int q) {
    return lca(root, p, q).val;
}

private Node lca(Node n, int p, int q) {
    if (n == null || n.val == p || n.val == q) return n;
    Node l = lca(n.left, p, q);
    Node r = lca(n.right, p, q);
    if (l != null && r != null) return n;
    return l != null ? l : r;
}

```

Each call returns p , q , their LCA, or `null`. If both subtrees report something, p and q are on different sides, so this node is the LCA. **$O(n)$** .

4.9 Step 19: root-to-leaf paths and maximum path sum

```

BinaryTree.java  paths

public List<List<Integer>> rootToLeafPaths() {
    List<List<Integer>> out = new ArrayList<>();
    paths(root, new ArrayList<>(), out);
    return out;
}

private void paths(Node n, List<Integer> path, List<List<Integer>> out) {
    if (n == null) return;
    path.add(n.val); // choose
    if (n.left == null && n.right == null) out.add(new ArrayList<>(path));
    paths(n.left, path, out); // explore
    paths(n.right, path, out);
    path.remove(path.size() - 1); // un-choose (backtrack)
}

public int maxPathSum() { // any start, any end
    int[] best = {Integer.MIN_VALUE};
    gain(root, best);
    return best[0];
}

private int gain(Node n, int[] best) { // best downward path from n
    if (n == null) return 0;
    int l = Math.max(0, gain(n.left, best)); // drop negative branches
    int r = Math.max(0, gain(n.right, best));
    best[0] = Math.max(best[0], n.val + l + r);
    return n.val + Math.max(l, r);
}

```

- `paths` carries the current path **down** and uses **backtracking**: add the node, explore, remove the node. At a leaf, store a **copy** of the path.
- `maxPathSum`: the helper returns the best **downward** path from a node (so the parent can extend it) and records the best path **bending** at the node. `Math.max(0, ...)` ignores branches that would lower the sum.

4.10 Step 20: serialize and deserialize

Goal: convert a tree to a string and back (to save it in a file or send it over a network). Preorder with `#` for every `null` describes a tree **uniquely**, and reading it back in the same preorder rebuilds it:

```

BinaryTree.java  serialize

public String serialize() {                                // preorder, "#" for null
    StringBuilder sb = new StringBuilder();
    serialize(root, sb);
    return sb.substring(0, sb.length() - 1);              // drop the trailing comma
}

private void serialize(Node n, StringBuilder sb) {
    if (n == null) {
        sb.append("#,");
        return;
    }
    sb.append(n.val).append(',');
    serialize(n.left, sb);
    serialize(n.right, sb);
}

public static BinaryTree deserialize(String data) {
    BinaryTree tree = new BinaryTree();
    tree.root = decode(data.split(","), new int[]{0});
    return tree;
}

private static Node decode(String[] tokens, int[] i) {
    String t = tokens[i[0]++];
    if (t.equals("#")) return null;
    Node n = new Node(Integer.parseInt(t));
    n.left = decode(tokens, i);                            // same order as serialize
    n.right = decode(tokens, i);
    return n;
}

```

Checkpoint 10. Everything from steps 15 to 20:

```

BinaryTree.java  checkAlgorithms

System.out.println("balanced=" + t.isBalanced() + " diameter=" + t.diameter()
    + " symmetric=" + t.isSymmetric());
System.out.println("lca(7,4)=" + t.lca(7, 4) + " lca(7,6)=" + t.lca(7, 6)
    + " lca(8,5)=" + t.lca(8, 5));
System.out.println("paths: " + t.rootToLeafPaths() + " maxPathSum=" + t.maxPathSum());
String s = t.serialize();
System.out.println("serialized: " + s);
BinaryTree back = BinaryTree.deserialize(s);
back.mirror();
System.out.println("deserialized + mirrored: " + back.shape());
BinaryTree chain = BinaryTree.fromLevelOrder(1, null, 2, null, 3);
System.out.println("chain 1-2-3 balanced? " + chain.isBalanced());

```

>_ CHECKPOINT OUTPUT: BINARYTREE.JAVA, ALGORITHMS (ACTUAL RUN)

```

balanced=true diameter=5 symmetric=false
lca(7,4)=2 lca(7,6)=1 lca(8,5)=5
paths: [[1, 2, 4], [1, 2, 5, 7], [1, 2, 5, 8], [1, 3, 6]] maxPathSum=25
serialized: 1,2,4,#,#,5,7,#,#,8,#,#,3,#,6,#,#
deserialized + mirrored: 1(3(6,),2(5(8,7),4))
chain 1-2-3 balanced? false

```

4.11 Your finished BinaryTree

You've written the whole class. Compare your file with `trees-java/BinaryTree.java`.

Method	What it does	Time
<code>isEmpty</code> , <code>clear</code>	state of the tree	O(1)
<code>insert(v)</code>	first free level-order slot	O(n)
<code>fromLevelOrder(...)</code>	build any shape (factory)	O(n)
<code>print</code> , <code>shape</code>	draw the tree	O(n)
<code>preorder</code> , <code>inorder</code> , <code>postorder</code> (+ iterative)	depth-first orders	O(n)
<code>levelOrder</code> , <code>zigzag</code> , views	breadth-first orders	O(n)
<code>morrisInorder</code>	inorder with O(1) memory	O(n)
<code>fromPreIn(pre, in)</code>	rebuild from two traversals	O(n)
<code>size</code> , <code>height</code> , <code>countLeaves</code>	measurements	O(n)
<code>contains</code> , <code>depth</code> , <code>min</code> , <code>max</code>	search (no ordering)	O(n)
<code>delete(v)</code>	swap with deepest node, remove it	O(n)
<code>isFull</code> , <code>isComplete</code> , <code>isPerfect</code> , <code>isBalanced</code>	shape checks	O(n)
<code>diameter</code> , <code>lca</code> , <code>rootToLeafPaths</code> , <code>maxPathSum</code>	classic algorithms	O(n)
<code>mirror</code> , <code>isSymmetric</code>	mirror operations	O(n)
<code>serialize</code> , <code>deserialize</code>	save / load	O(n)

Σ CHAPTER SUMMARY

- Template: **null base case, recurse left and right, combine**. Information flows up (height, size) or down (depth, paths).
- `delete` in a plain binary tree: copy the **deepest, rightmost** value into the target and remove that leaf.
- Sentinels (`UNBALANCED`) and “return one thing, record another” turn O(n²) ideas into O(n).
- Almost every operation on a plain binary tree is O(n), because nothing tells us where a value is. The next chapter fixes that with an **ordering rule**.

P A R T

II

Search Trees

We build a `BinarySearchTree` and then an `AVLTree` step by step, then meet red-black, splay, treap, B-tree and B+ tree.

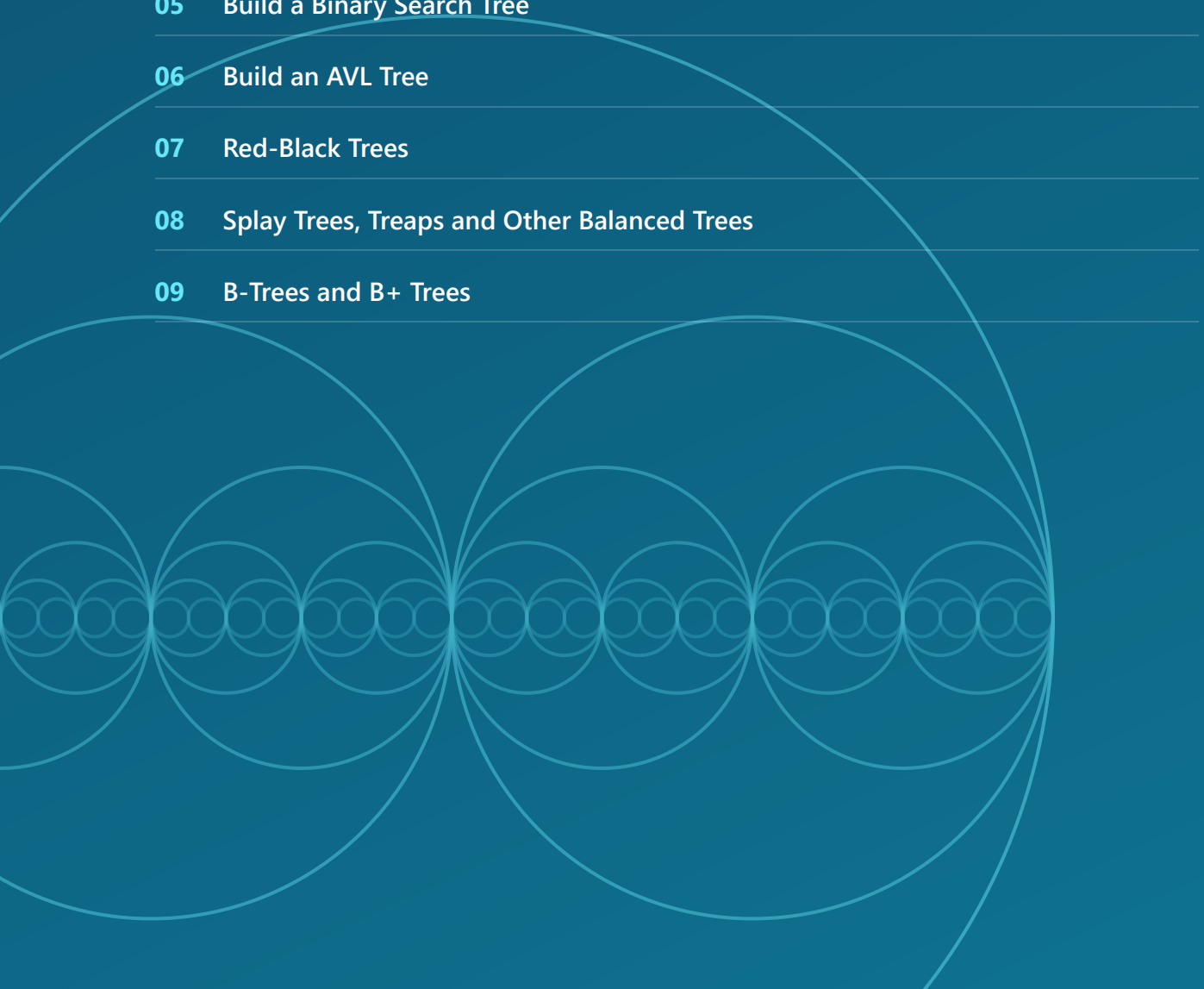
05 Build a Binary Search Tree

06 Build an AVL Tree

07 Red-Black Trees

08 Splay Trees, Treaps and Other Balanced Trees

09 B-Trees and B+ Trees



CHAPTER 5

Build a Binary Search Tree

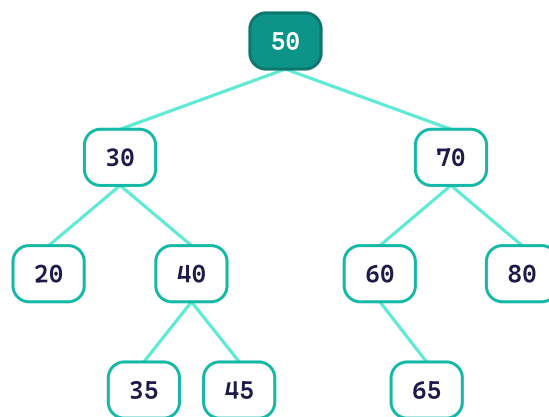
05

5.1 The one rule that changes everything

In our `BinaryTree`, finding a value meant searching **everywhere**: $O(n)$. A **binary search tree (BST)** adds one rule, which must hold at **every** node:

★ THE BST PROPERTY

Every key in a node's **left** subtree is **smaller** than the node's key, and every key in its **right** subtree is **larger**.



A binary search tree: $left < node < right$, at every node

■ root

Now, at every node, one comparison tells us which half to throw away, exactly like binary search on a sorted array. Unlike a sorted array, though, a BST also inserts and deletes quickly, because adding a node only changes one pointer. Two consequences to remember:

- Search, insert, delete, min and max all follow **one path** from the root: **$O(h)$** .
- **Inorder traversal visits the keys in sorted order** (left = smaller, then the node, then right = larger).

We'll now write `BinarySearchTree.java` from scratch, in the same step/checkpoint format as `BinaryTree`.

5.2 Step 1: The skeleton, with a size counter

```

BinarySearchTree.java  skeleton

private static class Node {
    int key;
    Node left, right;

    Node(int key) {
        this.key = key;
    }
}

private Node root;
private int count;           // number of keys, kept up to date by insert/delete

public boolean isEmpty() {
    return root == null;
}

public int size() {          // O(1) thanks to the counter
    return count;
}

```

Same `Node` and `root` as before (the value is called `key` because it is used for ordering). One improvement: a `count` field kept up to date by `insert` and `delete`, so `size()` is **O(1)** instead of an $O(n)$ traversal. This is a common design trade-off: a tiny bit of extra state for a much faster query.

5.3 Step 2: insert (recursive)

Goal: put a new key in the one position where the BST property still holds. Walk down as if searching for the key; when you fall off the tree at a `null` link, that's exactly where the new node belongs.

? TRY IT FIRST

Write a private `Node insert(Node node, int key)` that **returns the root of the subtree after insertion**. Base case: `node == null` means return a new node. Otherwise recurse left or right and **re-assign** the child link.

```

BinarySearchTree.java  insert

public boolean insert(int key) {
    if (contains(key)) return false; // no duplicates in this BST
    root = insert(root, key);
    count++;
    return true;
}

// Returns the root of the subtree after inserting key into it.
private Node insert(Node node, int key) {
    if (node == null) return new Node(key); // fell off the tree: the key goes HERE
    if (key < node.key) {
        node.left = insert(node.left, key); // re-link: the child may be a new node
    } else {
        node.right = insert(node.right, key);
    }
    return node;
}

```

- The “return the new subtree root” pattern is the key idea. `node.left = insert(node.left, key)` works whether or not the child changed: if `node.left` was `null`, the call returns the new node and the assignment attaches it; otherwise it returns the same child and the assignment changes nothing. The same pattern makes `delete` and the AVL rotations easy later.
- Duplicates are rejected (`contains` first), so `count` stays correct. Some BSTs allow duplicates (by sending equal keys right) or store a count per node instead.
- New keys always become **leaves**, so the shape of a BST depends on the **order** of insertion.

Complexity: $O(h)$.

5.4 Step 3: insertIterative

The same algorithm as a loop, which is what production code usually does (no recursion depth to worry about). Walk down; when the chosen child is `null`, attach the new node there.

```

BinarySearchTree.java insertIterative

public boolean insertIterative(int key) {
    Node fresh = new Node(key);
    if (root == null) {
        root = fresh;
        count++;
        return true;
    }
    Node cur = root;
    while (true) {
        if (key == cur.key) return false; // duplicate
        if (key < cur.key) {
            if (cur.left == null) { cur.left = fresh; break; }
            cur = cur.left;
        } else {
            if (cur.right == null) { cur.right = fresh; break; }
            cur = cur.right;
        }
    }
    count++;
    return true;
}

```

5.5 Step 4: contains, inorder, print

```

BinarySearchTree.java contains

public boolean contains(int key) {
    Node cur = root;
    while (cur != null) {
        if (key == cur.key) return true;
        cur = key < cur.key ? cur.left : cur.right; // throw away one whole subtree
    }
    return false;
}

```

Search is the BST idea in its purest form: compare, then go left or right. Each step goes one level down: **O(h)** instead of **O(n)**.

```

BinarySearchTree.java print

public List<Integer> inorder() { // sorted order!
    List<Integer> out = new ArrayList<>();
    inorder(root, out);
    return out;
}

private void inorder(Node n, List<Integer> out) {
    if (n == null) return;
    inorder(n.left, out);
}

```

```

        out.add(n.key);
        inorder(n.right, out);
    }

    public String shape() {
        return shape(root);
    }

    private String shape(Node n) {
        if (n == null) return "";
        if (n.left == null && n.right == null) return String.valueOf(n.key);
        return n.key + "(" + shape(n.left) + "," + shape(n.right) + ")";
    }

    public void print() {
        StringBuilder sb = new StringBuilder();
        if (root == null) sb.append("(empty tree)\n");
        else print(root, "", sb.append(root.key).append('\n'));
        System.out.print(sb);
    }

    private StringBuilder print(Node n, String prefix, StringBuilder sb) {
        Node[] kids = {n.left, n.right};
        String[] sides = {"L", "R"};
        int shown = (n.left != null ? 1 : 0) + (n.right != null ? 1 : 0);
        for (int i = 0; i < 2; i++) {
            if (kids[i] == null) continue;
            boolean last = --shown == 0;
            sb.append(prefix).append(last ? "└─ " : "├─ ")
              .append(sides[i]).append(": ").append(kids[i].key).append('\n');
            print(kids[i], prefix + (last ? "    " : "│  "), sb);
        }
        return sb;
    }
}

```

`inorder` returns the keys **sorted**. `print` and `shape` are the same as in `BinaryTree` (the printer is written slightly differently, with arrays for the two children: compare the two approaches).

Checkpoint 1.

BinarySearchTree.java checkBuild

```

BinarySearchTree bst = new BinarySearchTree();
for (int k : new int[]{50, 30, 70, 20, 40, 60, 80, 35, 45, 65}) bst.insert(k);
bst.print();
System.out.println("size=" + bst.size() + " insert 40 again? " + bst.insert(40)
    + " insertIterative 55? " + bst.insertIterative(55) + " size=" + bst.size());
System.out.println("inorder: " + bst.inorder());

```

>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, BUILD (ACTUAL RUN)

```

50
├─ L: 30
│   ├── L: 20
│   └─ R: 40
│       ├── L: 35
│       └─ R: 45
└─ R: 70
    ├── L: 60
    └─ R: 80
    
```

size=10 insert 40 again? false insertIterative 55? true size=11
inorder: [20, 30, 35, 40, 45, 50, 55, 60, 65, 70, 80]

Inserting 40 again returns `false` and the size doesn't change. The inorder list is sorted, as promised.

5.6 Step 5: min, max, height

Goal: the minimum is the **leftmost** node, the maximum the **rightmost**. No recursion needed: follow one pointer until it runs out.

```

BinarySearchTree.java minMax

public int min() {
    if (root == null) throw new IllegalStateException("empty tree");
    Node cur = root;
    while (cur.left != null) cur = cur.left;    // keep going left
    return cur.key;
}

public int max() {
    if (root == null) throw new IllegalStateException("empty tree");
    Node cur = root;
    while (cur.right != null) cur = cur.right;  // keep going right
    return cur.key;
}

```

```

BinarySearchTree.java height

public int height() {
    // edges; empty = -1
    return height(root);
}

private int height(Node n) {
    if (n == null) return -1;
    return 1 + Math.max(height(n.left), height(n.right));
}

```

`min` / `max` are **O(h)** (compare with $O(n)$ in the plain binary tree). `height` still has to look at every node: $O(n)$.

Checkpoint 2.

BinarySearchTree.java checkSearch

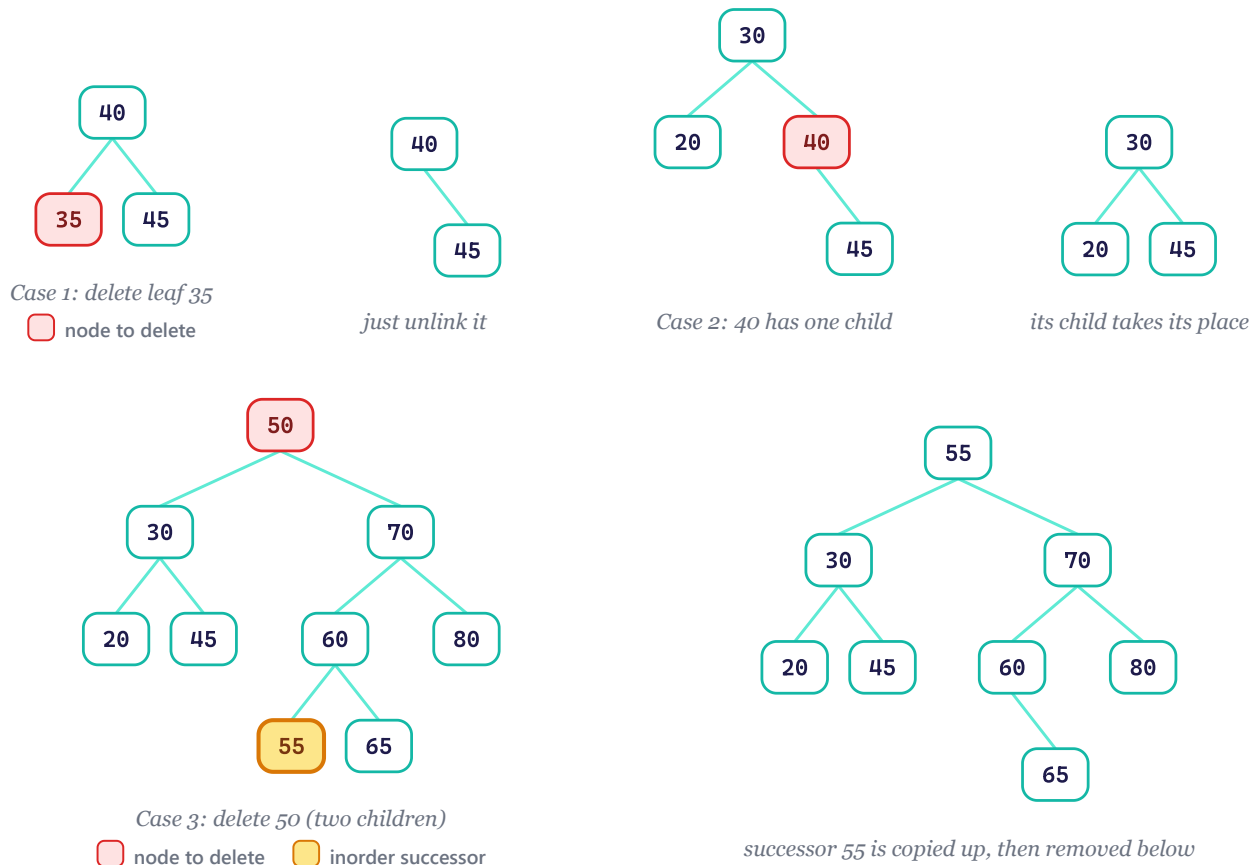
```
System.out.println("contains(45)=" + bst.contains(45) + " contains(99)=" + bst.contains(99));
System.out.println("min=" + bst.min() + " max=" + bst.max() + " height=" + bst.height());
```

>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, SEARCH (ACTUAL RUN)

```
contains(45)=true contains(99)=false
min=20 max=80 height=3
```

5.7 Step 6: delete, the three cases

Goal: remove a key and keep the BST property. Find the node, then:



- 1. Leaf:** just unlink it; the parent's link becomes `null`.
- 2. One child:** replace the node with its only child. The child's whole subtree was already on the correct side of the parent, so order is preserved.
- 3. Two children:** we can't just unlink it. Find the **inorder successor**, the smallest key in the right subtree (go right once, then left as far as possible). It's the next larger key, so it can take this node's place: everything on the left is smaller, everything on the right is larger. Copy its key here, then delete the successor from the right subtree. The successor has **no left child** (otherwise that child would be smaller), so that second delete is always case 1 or 2.

? TRY IT FIRST

Use the same “return the new subtree root” pattern as insert. Cases 1 and 2 can be written together: if

`node.left == null`, return `node.right` (which may itself be `null`).

```
BinarySearchTree.java delete

public boolean delete(int key) {
    if (!contains(key)) return false;
    root = delete(root, key);
    count--;
    return true;
}

// Returns the root of the subtree after removing key from it.
private Node delete(Node node, int key) {
    if (key < node.key) {
        node.left = delete(node.left, key);
    } else if (key > node.key) {
        node.right = delete(node.right, key);
    } else {
        // found the node to remove
        if (node.left == null) return node.right; // case 1 (leaf) and case 2 (right child
only)
        if (node.right == null) return node.left; // case 2 (left child only)
        Node succ = node.right; // case 3: two children
        while (succ.left != null) succ = succ.left; // inorder successor = min of right
subtree
        node.key = succ.key; // copy it here
        node.right = delete(node.right, succ.key); // remove the original (it has no left
child)
    }
    return node;
}
```

Notice how the return-the-subtree pattern makes cases 1 and 2 a single `return`: returning a child (or `null`) is the unlinking, because the caller assigns the result to its link. Using the inorder **predecessor** (the maximum of the left subtree) is equally correct.

Checkpoint 3: one deletion of each kind, including the root:

```

BinarySearchTree.java  checkDelete

System.out.println("start                : " + bst.shape());
bst.delete(35);
System.out.println("delete 35 (leaf)      : " + bst.shape());
bst.delete(40);
System.out.println("delete 40 (one child) : " + bst.shape());
bst.delete(50);
System.out.println("delete 50 (two children,root): " + bst.shape());
System.out.println("delete 99 (missing)    : " + bst.delete(99) + ", size=" +
bst.size());

```

>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, DELETE (ACTUAL RUN)

```

start                : 50(30(20,40(35,45)),70(60(55,65),80))
delete 35 (leaf)      : 50(30(20,40(,45)),70(60(55,65),80))
delete 40 (one child) : 50(30(20,45),70(60(55,65),80))
delete 50 (two children,root): 55(30(20,45),70(60(,65),80))
delete 99 (missing)   : false, size=8

```

Complexity: $O(h)$.

5.8 Step 7: floor, ceil, successor, predecessor

Goal: “nearest key” queries, the reason ordered maps exist (“the closest price at or below 52”). Walk one path and remember the best candidate seen:

```

BinarySearchTree.java floorCeil

public Integer floor(int key) {                                // largest key <= key, or null
    Integer best = null;
    Node cur = root;
    while (cur != null) {
        if (cur.key == key) return key;
        if (cur.key < key) {
            best = cur.key;                                     // candidate; a bigger one may be right
            cur = cur.right;
        } else {
            cur = cur.left;
        }
    }
    return best;
}

public Integer ceil(int key) {                                  // smallest key >= key, or null
    Integer best = null;
    Node cur = root;
    while (cur != null) {
        if (cur.key == key) return key;
        if (cur.key > key) {
            best = cur.key;                                     // candidate; a smaller one may be left
            cur = cur.left;
        } else {
            cur = cur.right;
        }
    }
    return best;
}

```

For `floor(key)`: when a node's key is smaller than the target, it's a candidate, but a closer one may exist in its **right** subtree, so go right. When it's larger, it can't be the answer, so go left. `ceil` is the mirror image.

```

BinarySearchTree.java succPred

public Integer successor(int key) {                // smallest key > key
    Integer best = null;
    Node cur = root;
    while (cur != null) {
        if (cur.key > key) {
            best = cur.key;
            cur = cur.left;
        } else {
            cur = cur.right;
        }
    }
    return best;
}

public Integer predecessor(int key) {              // largest key < key
    Integer best = null;
    Node cur = root;
    while (cur != null) {
        if (cur.key < key) {
            best = cur.key;
            cur = cur.right;
        } else {
            cur = cur.left;
        }
    }
    return best;
}

```

Successor and predecessor are the strict versions (`>` and `<`), and they work even if the key itself isn't in the tree. All four: **O(h)**; they return `null` when no such key exists.

Checkpoint 4.

```

BinarySearchTree.java checkNavigate

System.out.println("keys: " + bst.inorder());
System.out.println("floor(52)=" + bst.floor(52) + " ceil(52)=" + bst.ceil(52)
    + " floor(10)=" + bst.floor(10) + " ceil(90)=" + bst.ceil(90));
System.out.println("successor(45)=" + bst.successor(45) + " predecessor(50)="
    + bst.predecessor(50) + " successor(80)=" + bst.successor(80));

```

>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, NAVIGATE (ACTUAL RUN)

```

keys: [20, 30, 45, 55, 60, 65, 70, 80]
floor(52)=45 ceil(52)=55 floor(10)=null ceil(90)=null
successor(45)=55 predecessor(50)=45 successor(80)=null

```

5.9 Step 8: kthSmallest, range, isValid, lca

```

BinarySearchTree.java kth

public int kthSmallest(int k) {                                // 1-based
    Deque<Node> stack = new ArrayDeque<>();
    Node cur = root;
    while (cur != null || !stack.isEmpty()) {
        while (cur != null) {
            stack.push(cur);
            cur = cur.left;
        }
        cur = stack.pop();
        if (--k == 0) return cur.key;                        // the k-th node in inorder
        cur = cur.right;
    }
    throw new IllegalArgumentException("k > size");
}

```

The k-th smallest key is the k-th node visited in inorder: the iterative inorder from the binary tree chapter, stopping early. **$O(h + k)$** .

```

BinarySearchTree.java range

public List<Integer> range(int lo, int hi) {                 // all keys in [lo, hi], sorted
    List<Integer> out = new ArrayList<>();
    range(root, lo, hi, out);
    return out;
}

private void range(Node n, int lo, int hi, List<Integer> out) {
    if (n == null) return;
    if (lo < n.key) range(n.left, lo, hi, out);              // skip left if everything there < lo
    if (lo <= n.key && n.key <= hi) out.add(n.key);
    if (n.key < hi) range(n.right, lo, hi, out);            // skip right if everything there > hi
}

```

A range query **prunes**: it only enters the left subtree if keys there could be $\geq$ `lo`, and only the right subtree if keys there could be $\leq$ `hi`. **$O(h + \text{number of results})$** .

```

BinarySearchTree.java validate

public boolean isValid() {
    return isValid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

private boolean isValid(Node n, long low, long high) {
    if (n == null) return true;
    if (n.key <= low || n.key >= high) return false; // outside the allowed window
    return isValid(n.left, low, n.key) && isValid(n.right, n.key, high);
}

```

❌ COMMON MISTAKE: VALIDATING BY COMPARING WITH THE CHILDREN

Checking only `left.key < node.key < right.key` at each node is **wrong**: a key deep in the left subtree could still be larger than an ancestor. Pass down an allowed window `(low, high)`: going left lowers the upper bound, going right raises the lower bound.

```

BinarySearchTree.java lca

public int lca(int p, int q) {
    // both keys must exist
    Node cur = root;
    while (true) {
        if (p < cur.key && q < cur.key) cur = cur.left;
        else if (p > cur.key && q > cur.key) cur = cur.right;
        else return cur.key; // p and q split here
    }
}

```

In a BST, the LCA is the first node whose key lies **between** p and q . While both are smaller, go left; while both are larger, go right. **O(h)**, where the plain binary tree needed $O(n)$.

Checkpoint 5.

```

BinarySearchTree.java checkQueries

System.out.println("3rd smallest=" + bst.kthSmallest(3) + " range[40,66]=" + bst.range(40,
66));
System.out.println("lca(20,45)=" + bst.lca(20, 45) + " lca(20,80)=" + bst.lca(20, 80)
+ " isValid=" + bst.isValid());

```

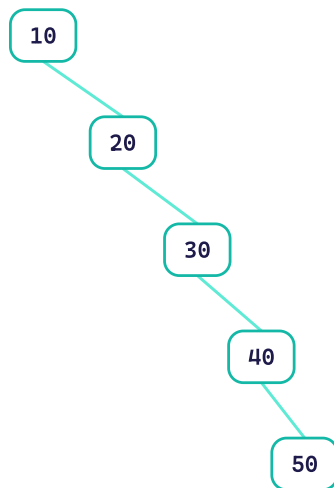
>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, QUERIES (ACTUAL RUN)

```

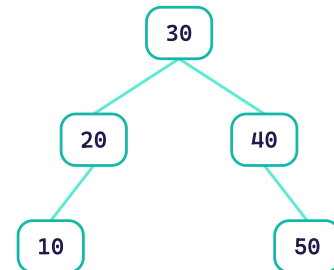
3rd smallest=45 range[40,66]=[45, 55, 60, 65]
lca(20,45)=30 lca(20,80)=55 isValid=true

```

5.10 Step 9: the weakness, and fromSorted



Keys inserted in sorted order



Same keys, balanced

Insert keys in **sorted order** and every key becomes the right child of the previous one: the tree degenerates into a linked list with height $n - 1$, and every $O(h)$ operation becomes **$O(n)$** . Real data often arrives sorted (IDs, timestamps). If the keys are known in advance, a perfectly balanced BST can be built by making the **middle** element the root, recursively:

```

BinarySearchTree.java  fromSorted

public static BinarySearchTree fromSorted(int[] sorted) {
    BinarySearchTree t = new BinarySearchTree();
    t.root = build(sorted, 0, sorted.length - 1);
    t.count = sorted.length;
    return t;
}

private static Node build(int[] a, int lo, int hi) {
    if (lo > hi) return null;
    int mid = (lo + hi) >>> 1;           // middle element becomes the root
    Node n = new Node(a[mid]);
    n.left = build(a, lo, mid - 1);
    n.right = build(a, mid + 1, hi);
    return n;
}
  
```

Checkpoint 6.

```

BinarySearchTree.java  checkSkewed

int n = 10_000;
BinarySearchTree skewed = new BinarySearchTree();
int[] keys = new int[n];
for (int k = 0; k < n; k++) {
    keys[k] = k + 1;
    skewed.insertIterative(k + 1);           // sorted input: worst case
}
BinarySearchTree balanced = BinarySearchTree.fromSorted(keys);
System.out.println("10,000 sorted inserts -> height " + skewed.height()
    + " | fromSorted -> height " + balanced.height());

```

>_ CHECKPOINT OUTPUT: BINARYSEARCHTREE.JAVA, SKEWED (ACTUAL RUN)

```
10,000 sorted inserts -> height 9999 | fromSorted -> height 13
```

A height of 9,999 means a search may take 10,000 steps instead of 14. When keys arrive one at a time, we need a tree that **rebalances itself**. That is the AVL tree, next.

Method	Average	Worst (degenerate)
insert, contains, delete	$O(\log n)$	$O(n)$
min, max, floor, ceil, successor, predecessor, lca	$O(\log n)$	$O(n)$
kthSmallest(k), range (k results)	$O(\log n + k)$	$O(n)$
size	$O(1)$	$O(1)$
inorder, height, isValid	$O(n)$	$O(n)$

Σ CHAPTER SUMMARY

- BST property: $\text{left} < \text{node} < \text{right}$, **at every node**; inorder traversal gives sorted keys.
- The **return-the-new-subtree-root** pattern makes insert and delete short and correct.
- Delete: leaf $\rightarrow$ unlink; one child $\rightarrow$ replace with the child; two children $\rightarrow$ copy the **inorder successor**, then delete it.
- Everything is **$O(h)$** , and sorted input makes $h = n - 1$. Balancing is needed.

CHAPTER 6

Build an AVL Tree

06

6.1 The idea

An **AVL tree** (Adelson-Velsky and Landis, 1962: the first self-balancing BST) is our BST plus one extra rule at every node:

★ THE AVL RULE

The heights of the left and right subtrees differ by **at most 1**. The **balance factor** `bf = height(left) - height(right)` must always be -1 , 0 or $+1$.

Whenever an insert or delete breaks the rule, the tree immediately repairs itself with one or two **rotations**. The result is that the height is always at most about $1.44 \cdot \log_2 n$. Why? The sparsest possible AVL tree of height h has $N(h) = 1 + N(h-1) + N(h-2)$ nodes, which grows like the Fibonacci numbers (1, 2, 4, 7, 12, 20, 33 ...): exponentially in h . So h can only be logarithmic in n .

We'll build `AVLTree.java` by **copying the BST** and changing only what's needed. You'll see that the entire difference is one field, three helpers, two rotations, one `rebalance` method, and **one extra line** in `insert` and `delete`.

6.2 Step 1: The skeleton, with a height in every node

```

AVLTree.java  skeleton

private static class Node {
    int key;
    int height;           // NEW: height in edges (a new leaf has 0)
    Node left, right;

    Node(int key) {
        this.key = key;
    }
}

private Node root;
private int count;
private final List<String> rotations = new ArrayList<>(); // what the last operation did

public int size() {
    return count;
}

public List<String> lastRotations() {
    return rotations;
}

```

The only change to `Node` is the new `height` field. Computing heights recursively each time would cost $O(n)$; storing them makes them **$O(1)$** to read. The `rotations` list is just for learning: it records which rotations the last operation performed, so the checkpoints can show them.

`contains` and `shape` are copied unchanged from the BST, because searching never changes a tree:

```

AVLTree.java fromBst

// Unchanged from the BST: search never modifies the tree.
public boolean contains(int key) {
    Node cur = root;
    while (cur != null) {
        if (key == cur.key) return true;
        cur = key < cur.key ? cur.left : cur.right;
    }
    return false;
}

public String shape() {
    return shape(root);
}

private String shape(Node n) {
    if (n == null) return "";
    if (n.left == null && n.right == null) return String.valueOf(n.key);
    return n.key + "(" + shape(n.left) + "," + shape(n.right) + ")";
}

```

6.3 Step 2: height, balance and update helpers

```

AVLTree.java helpers

private int height(Node n) {
    return n == null ? -1 : n.height;    // 0(1): read the stored value
}

private int balance(Node n) {           // balance factor: must stay in {-1, 0, +1}
    return n == null ? 0 : height(n.left) - height(n.right);
}

private void update(Node n) {           // call whenever n's children change
    n.height = 1 + Math.max(height(n.left), height(n.right));
}

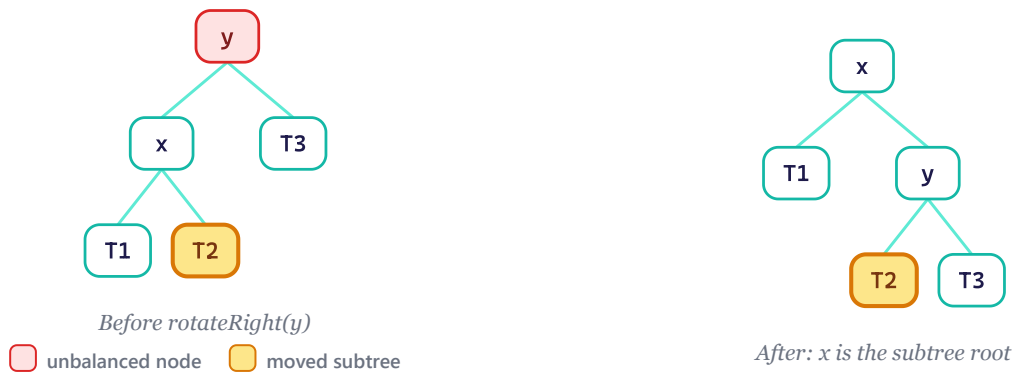
public int height() {
    return height(root);
}

```

- `height(null)` is `-1`, so a new leaf (height 0) is consistent.
- `update(n)` recomputes a node's height from its children's **stored** heights. It must be called whenever a node's children change: after inserting below it, after deleting below it, and during rotations.

6.4 Step 3: rotations

A **right rotation** at `y` lifts its left child `x` up; `y` becomes `x`'s right child, and `x`'s old right subtree `T2` moves across to become `y`'s left subtree:



The inorder order is **T1, x, T2, y, T3** before and after, so the BST property is preserved. **T2** holds keys between **x** and **y**, so it can hang on either side. Only three links change: **O(1)**.

? TRY IT FIRST

Write `rotateRight(Node y)` returning the new subtree root. You need to save `x = y.left` and `t2 = x.right`, change two links, update two heights (in which order?), and return `x`.

AVLTree.java rotations

```
//      y          x
//     / \        / \
//    x  T3  ==> T1  y
//   / \      / \
//  T1 T2    T2 T3
private Node rotateRight(Node y) {
    Node x = y.left;
    Node t2 = x.right;
    x.right = y;
    y.left = t2;
    update(y);
    update(x);
    return x;
}

private Node rotateLeft(Node x) {
    Node y = x.right;
    Node t2 = y.left;
    y.left = x;
    x.right = t2;
    update(x);
    update(y);
    return y;
}
```

// y moves down to the right
// T2 changes parent (keys between x and y)
// y is now below x: update it first
// new root of this subtree

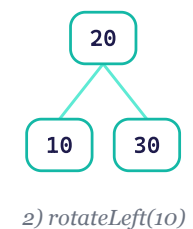
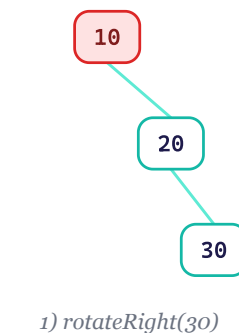
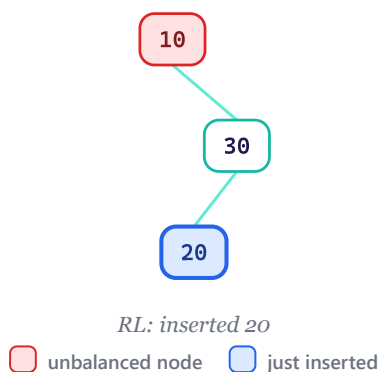
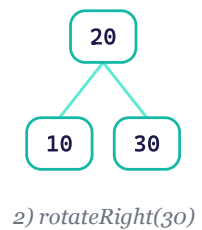
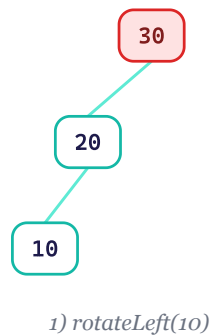
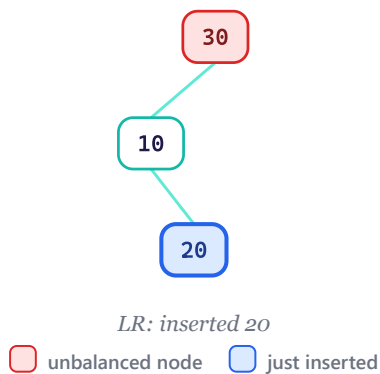
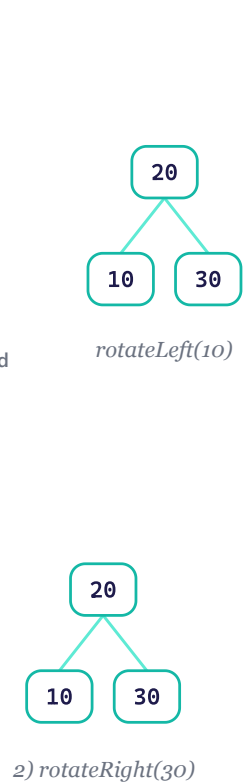
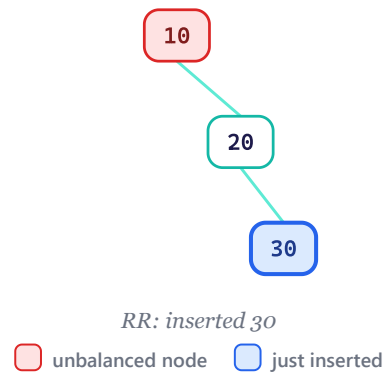
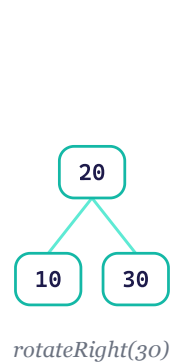
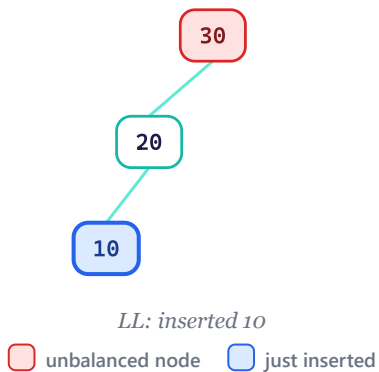
// exact mirror image

! UPDATE THE LOWER NODE FIRST

After the rotation, y is x 's child. Update y first, then x , because x 's new height depends on y 's new height.

6.5 Step 4: rebalance, the four cases

After an insert or delete, walk back up the path. At the first node where $|bf| = 2$, the **shape** of the imbalance decides the fix. The case names describe the path from the unbalanced node down toward the tall side:



Case	Condition	Fix
LL	$\text{bf}(\text{node}) = +2$ and $\text{bf}(\text{left}) \geq 0$	<code>rotateRight(node)</code>
LR	$\text{bf}(\text{node}) = +2$ and $\text{bf}(\text{left}) < 0$	<code>rotateLeft(left)</code> , then <code>rotateRight(node)</code>
RR	$\text{bf}(\text{node}) = -2$ and $\text{bf}(\text{right}) \leq 0$	<code>rotateLeft(node)</code>
RL	$\text{bf}(\text{node}) = -2$ and $\text{bf}(\text{right}) > 0$	<code>rotateRight(right)</code> , then <code>rotateLeft(node)</code>

A single rotation fixes a **straight line** (LL, RR). A **zig-zag** (LR, RL) first needs a rotation at the child to straighten it into a line.

```

AVLTree.java  rebalance

private Node rebalance(Node n) {
    update(n);
    int bf = balance(n);
    if (bf > 1) {
        // left side too tall
        if (balance(n.left) < 0) {
            // LR: left child leans right
            rotations.add("LR at " + n.key);
            n.left = rotateLeft(n.left);
            // straighten it into LL
        } else {
            rotations.add("LL at " + n.key);
        }
        return rotateRight(n);
    }
    if (bf < -1) {
        // right side too tall
        if (balance(n.right) > 0) {
            // RL: right child leans left
            rotations.add("RL at " + n.key);
            n.right = rotateRight(n.right);
            // straighten it into RR
        } else {
            rotations.add("RR at " + n.key);
        }
        return rotateLeft(n);
    }
    return n;
    // balanced: nothing to do
}

```

`rebalance` always starts with `update(n)`, so it also takes care of keeping the stored heights correct.

6.6 Step 5: insert, the BST insert plus one line

? TRY IT FIRST

Copy your BST's recursive insert. What single change makes it an AVL insert?

AVLTree.java insert

```

public boolean insert(int key) {
    rotations.clear();
    if (contains(key)) return false;
    root = insert(root, key);
    count++;
    return true;
}

private Node insert(Node node, int key) {
    if (node == null) return new Node(key);           // same as BST insert...
    if (key < node.key) node.left = insert(node.left, key);
    else node.right = insert(node.right, key);
    return rebalance(node);                          // ...plus ONE line on the way up
}

```

The only change: instead of `return node;` we `return rebalance(node);`. The recursion unwinds from the new leaf up to the root, so **every ancestor** gets its height updated and, if needed, rotated, and the rotated subtree's new root is re-linked to its parent by the same return-the-subtree pattern. After an insertion, at most **one** (single or double) rotation actually happens, because it restores the subtree's original height.

Checkpoint 1: each case on three keys, then the classic sequence 10, 20, 30, 40, 50, 25:

AVLTree.java checkCases

```

int[][] orders = {{30, 20, 10}, {10, 20, 30}, {30, 10, 20}, {10, 30, 20}};
for (int[] order : orders) {
    AVLTree t = new AVLTree();
    for (int k : order) t.insert(k);
    System.out.println(java.util.Arrays.toString(order) + " -> " + t.lastRotations()
        + " -> " + t.shape());
}

```

>_ CHECKPOINT OUTPUT: AVLTREE.JAVA, CASES (ACTUAL RUN)

```

[30, 20, 10] -> [LL at 30] -> 20(10,30)
[10, 20, 30] -> [RR at 10] -> 20(10,30)
[30, 10, 20] -> [LR at 30] -> 20(10,30)
[10, 30, 20] -> [RL at 10] -> 20(10,30)

```

AVLTree.java checkSequence

```

AVLTree avl = new AVLTree();
for (int k : new int[]{10, 20, 30, 40, 50, 25}) trace(avl, "insert", k, true);

```

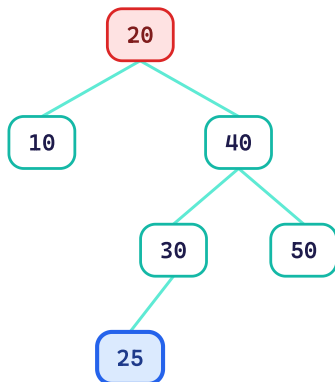
>_ CHECKPOINT OUTPUT: AVLTREE.JAVA, SEQUENCE (ACTUAL RUN)

```

insert 10  rotation: -      -> 10
insert 20  rotation: -      -> 10(,20)
insert 30  rotation: RR at 10 -> 20(10,30)
insert 40  rotation: -      -> 20(10,30(,40))
insert 50  rotation: RR at 30 -> 20(10,40(30,50))
insert 25  rotation: RL at 20 -> 30(20(10,25),40(,50))

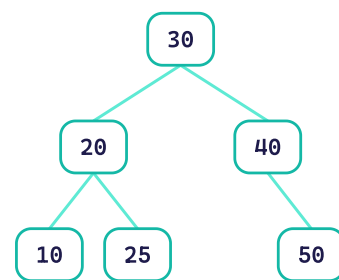
```

Inserting 25 makes node 20 unbalanced (bf = -2) with a right child (40) that leans left: an RL case. That is the pair of diagrams below.



insert 25: node 20 has balance -2

 unbalanced node just inserted



after the RL rotation at 20

6.7 Step 6: delete, also plus one line

```

AVLTree.java delete

public boolean delete(int key) {
    rotations.clear();
    if (!contains(key)) return false;
    root = delete(root, key);
    count--;
    return true;
}

private Node delete(Node node, int key) {
    if (key < node.key) {
        node.left = delete(node.left, key);
    } else if (key > node.key) {
        node.right = delete(node.right, key);
    } else {
        if (node.left == null) return node.right; // 0 or 1 child: same as BST
        if (node.right == null) return node.left;
        Node succ = node.right; // 2 children: same as BST
        while (succ.left != null) succ = succ.left;
        node.key = succ.key;
        node.right = delete(node.right, succ.key);
    }
    return rebalance(node); // every ancestor may need fixing
}

```

Again an unchanged BST delete, with `return rebalance(node)` at the end. One difference from insertion: after a delete, a rotation can make a subtree **shorter**, which may unbalance an ancestor further up, so a deletion can trigger rotations at **several** levels (at most one per level, $O(\log n)$ in total).

6.8 Step 7: an invariant checker

When writing balanced trees, a checker that verifies **every** invariant is the best debugging tool you can have. It checks BST order, the stored heights and the AVL rule at every node:

●●● AVLTree.java check

```
// Verifies BST order, stored heights and |balance| <= 1 everywhere.
public boolean isValidAVL() {
    try {
        check(root, Long.MIN_VALUE, Long.MAX_VALUE);
        return true;
    } catch (IllegalStateException e) {
        System.out.println("invalid: " + e.getMessage());
        return false;
    }
}

private int check(Node n, long low, long high) {
    if (n == null) return -1;
    if (n.key <= low || n.key >= high) throw new IllegalStateException("order at " + n.key);
    int l = check(n.left, low, n.key);
    int r = check(n.right, n.key, high);
    if (Math.abs(l - r) > 1) throw new IllegalStateException("balance at " + n.key);
    if (n.height != 1 + Math.max(l, r)) throw new IllegalStateException("height at " +
n.key);
    return n.height;
}
```

Checkpoint 2: deletions that trigger rotations, then a stress test (a million sorted inserts, the BST's worst case, then half of the keys deleted):

●●● AVLTree.java checkDelete

```
AVLTree d = new AVLTree();
for (int k : new int[]{9, 5, 10, 0, 6, 11, -1, 1, 2}) d.insert(k);
System.out.println("start                -> " + d.shape());
trace(d, "delete", 10, false);
trace(d, "delete", 11, false);
System.out.println("valid AVL? " + d.isValidAVL() + " size=" + d.size());
```

>_ CHECKPOINT OUTPUT: AVLTREE.JAVA, DELETE (ACTUAL RUN)

```
start                -> 9(1(0(-1,),5(2,6)),10(,11))
delete 10  rotation: LL at 9   -> 1(0(-1,),9(5(2,6),11))
delete 11  rotation: LL at 9   -> 1(0(-1,),5(2,9(6,)))
valid AVL? true  size=7
```

●●● AVLTree.java checkStress

```
AVLTree big = new AVLTree();
for (int k = 1; k <= 1_000_000; k++) big.insert(k);           // sorted: BST worst case
for (int k = 1; k <= 1_000_000; k += 2) big.delete(k);       // delete every odd key
System.out.println("size=" + big.size() + " height=" + big.height()
    + " valid=" + big.isValidAVL());
```

>_ CHECKPOINT OUTPUT: AVLTree.java, STRESS (ACTUAL RUN)

```
size=500000 height=18 valid=true
```

Half a million keys and a height of just 18, with every invariant verified, even though the input was sorted.

Operation	AVL tree
<code>contains</code> , <code>insert</code> , <code>delete</code>	$O(\log n)$ guaranteed
rotations per insert	at most 1 single or double
rotations per delete	$O(\log n)$
height	$\leq 1.44 \log_2 n$
extra memory	one <code>int</code> per node

✓ AVL VS. RED-BLACK

AVL trees are **more strictly balanced** ($1.44 \log n$ vs. $2 \log n$), so lookups are slightly faster. Red-black trees (next chapter) do **fewer rotations** on updates, which is why Java's `TreeMap` uses them.

Σ CHAPTER SUMMARY

- AVL = BST + $|bf| \leq 1$ everywhere. Height $\leq 1.44 \log_2 n$.
- Store the height in each node; `update` it whenever the children change.
- Rotations are $O(1)$ and preserve order. **LL/RR**: one rotation. **LR/RL**: two.
- AVL insert/delete = BST insert/delete with `return rebalance(node)`.

CHAPTER 7

Red-Black Trees



7.1 The rules

A **red-black tree** is a BST where each node is colored **red** or **black**, obeying:

1. Every node is red or black.
2. The **root is black**.
3. Every `null` leaf is considered black.
4. A red node never has a red child (no two reds in a row).
5. Every path from a node down to a `null` leaf contains the **same number of black nodes** (the **black height**).

Rules 4 and 5 together bound the height. The shortest possible root-to-leaf path is all black; the longest alternates red and black. Since both have the same number of black nodes, **the longest path is at most twice the shortest**, and the height is at most $2 \cdot \log_2(n + 1)$. The tree is less strictly balanced than AVL, but it needs fewer rotations to maintain.

7.2 Insertion in a classic red-black tree

A new node is inserted as in a BST and colored **red** (adding a black node would break rule 5 immediately). If its parent is black, we're done. If its parent is also red, rule 4 is broken, and the fix depends on the new node's **uncle** (its parent's sibling):

Case	Situation	Fix
1	uncle is red	recolor: parent and uncle black, grandparent red; continue fixing from the grandparent
2	uncle is black , node is an "inner" grandchild (zig-zag)	rotate at the parent to turn it into case 3
3	uncle is black , node is an "outer" grandchild (straight line)	rotate at the grandparent, swap the colors of parent and grandparent

Finally, color the root black. Case 1 can move up the tree ($O(\log n)$ recolorings), but **at most two rotations** happen per insert. Deletion has a similar but longer case analysis (up to three rotations).

7.3 Left-leaning red-black trees

The classic case analysis is long and error-prone, and it needs parent pointers. Robert Sedgwick's **left-leaning red-black tree (LLRB)** gets the same guarantees with a much shorter implementation. The trick is to think of a red node as **glued to its parent**: a black node with a red left child is a single node holding two keys. This makes a red-black tree an encoding of a **2-3 tree** (a B-tree where nodes hold 1 or 2 keys; see Chapter 9), which is perfectly balanced in terms of black links.

```

RedBlackTree.java  node

static final boolean RED = true, BLACK = false;

static class Node {
    int key;
    Node left, right;
    boolean color;           // color of the link from the parent to this node

    Node(int key) {
        this.key = key;
        this.color = RED;    // new nodes are always red
    }
}

static boolean isRed(Node n) {
    return n != null && n.color == RED;    // null links count as black
}

```

A node's color represents the color of the link from its parent. Three $O(1)$ operations maintain the invariants:

```

RedBlackTree.java  fixups

static Node rotateLeft(Node h) {           // turn a right-leaning red link left
    Node x = h.right;
    h.right = x.left;
    x.left = h;
    x.color = h.color;
    h.color = RED;
    return x;
}

static Node rotateRight(Node h) {          // turn a left-leaning red link right
    Node x = h.left;
    h.left = x.right;
    x.right = h;
    x.color = h.color;
    h.color = RED;
    return x;
}

static void flipColors(Node h) {           // split a temporary 4-node
    h.color = RED;
    h.left.color = BLACK;
    h.right.color = BLACK;
}

```

- `rotateLeft` turns a **right-leaning** red link into a left-leaning one.
- `rotateRight` is used temporarily when two red links appear in a row on the left.

- `flipColors` splits a temporary 4-node (both children red): the children become black and the red moves up to the parent, exactly like a B-tree node split pushing its middle key up.

7.4 Insert

```

RedBlackTree.java  insert

static Node root;

static void insert(int key) {
    root = insert(root, key);
    root.color = BLACK;           // the root is always black
}

static Node insert(Node h, int key) {
    if (h == null) return new Node(key);
    if (key < h.key) h.left = insert(h.left, key);
    else if (key > h.key) h.right = insert(h.right, key);

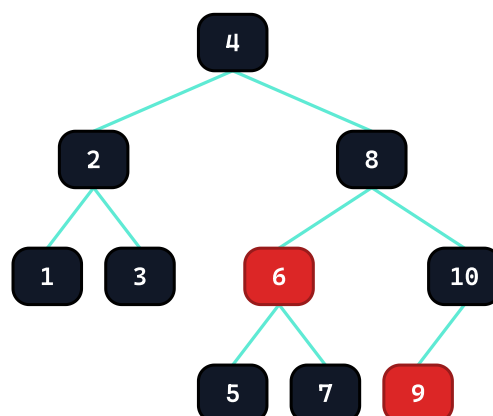
    if (isRed(h.right) && !isRed(h.left)) h = rotateLeft(h);    // lean left
    if (isRed(h.left) && isRed(h.left.left)) h = rotateRight(h); // no two reds in a row
    if (isRed(h.left) && isRed(h.right)) flipColors(h);         // push red upward
    return h;
}

```

The insert is a normal recursive BST insert (new nodes are red), followed by three checks **in this order** on the way back up:

1. Red link leaning right? Rotate left.
2. Two red links in a row on the left? Rotate right.
3. Both children red? Flip colors.

That's the whole algorithm. Deletion in LLRB trees is also possible but is more involved; in practice, library code (like `TreeMap`) implements the classic version.



Left-leaning red-black tree after inserting 1..10 (from the program output)

■ black ■ red

7.5 Verifying the rules

RedBlackTree.java validate

```
// Returns the black height if every red-black rule holds, otherwise -1.
static int blackHeight(Node h) {
    if (h == null) return 0;
    if (isRed(h) && (isRed(h.left) || isRed(h.right))) return -1; // red with red child
    int l = blackHeight(h.left), r = blackHeight(h.right);
    if (l == -1 || r == -1 || l != r) return -1; // unequal black paths
    return l + (isRed(h) ? 0 : 1);
}
```

>_ OUTPUT OF REDBLACKTREE.JAVA (ACTUAL RUN)

```
insert 1 -> 1
insert 2 -> 2(1*,)
insert 3 -> 2(1,3)
insert 4 -> 2(1,4(3*,))
insert 5 -> 4(2*(1,3),5)
insert 6 -> 4(2*(1,3),6(5*,))
insert 7 -> 4(2(1,3),6(5,7))
insert 8 -> 4(2(1,3),6(5,8(7*,)))
insert 9 -> 4(2(1,3),8(6*(5,7),9))
insert 10 -> 4(2(1,3),8(6*(5,7),10(9*,)))
black height = 3, height = 3
1..1,000,000 inserted in order: height = 19, black height = 19 (valid: true)
TreeMap: size=1000000 floorKey(500000)=500000
```

Inserting 1 to 10 in sorted order gives a tree of height 3; one million sorted keys give height 19 with every rule satisfied. The last line uses `java.util.TreeMap`, which is a red-black tree internally.

Property	AVL	Red-black
Balance rule	heights differ by ≤ 1	no red-red; equal black heights
Max height	$\approx 1.44 \log_2 n$	$\approx 2 \log_2 n$
Search	slightly faster	slightly slower
Rotations per insert	≤ 2	≤ 2
Rotations per delete	$O(\log n)$	≤ 3
Extra data per node	height (int)	color (1 bit)
Used in	databases, read-heavy lookups	<code>TreeMap</code> , <code>TreeSet</code> , <code>std::map</code> , Linux scheduler, <code>HashMap</code> buckets (Java 8+)

I RED-BLACK TREES INSIDE HASHMAP

Since Java 8, when many keys collide in one `HashMap` bucket (more than 8), the bucket's linked list is converted into a red-black tree, turning worst-case lookups in that bucket from $O(n)$ into $O(\log n)$.

Σ CHAPTER SUMMARY

- Five rules; the key ones: **no red node has a red child** and **all paths have the same black height**.
 $\text{Height} \leq 2 \log_2(n+1)$.
- Classic insert fixes red-red conflicts by **recoloring** (red uncle) or **rotating** (black uncle).
- LLRB: rotate left, rotate right, flip colors. It's a 2-3 tree in disguise.
- Java's `TreeMap` and `TreeSet` are red-black trees: guaranteed $O(\log n)$.

CHAPTER 8

Splay Trees, Treaps and Other Balanced Trees



AVL and red-black trees guarantee $O(\log n)$ for **every** operation. Other designs make different trade-offs.

8.1 Splay trees

A **splay tree** keeps no balance information at all. Instead, every access (search, insert, delete) **splays** the accessed node: rotates it all the way to the root. Recently used keys therefore end up near the top, which makes splay trees excellent for workloads with **locality** (caches, recently used items).

Splaying uses three kinds of steps, depending on the node x , its parent p and its grandparent g :

Step	Shape	Rotations
Zig	p is the root	one rotation at p
Zig-zig	x and p are both left children (or both right)	rotate at g first, then at p
Zig-zag	x is a right child and p a left child (or vice versa)	rotate at p , then at g (like an AVL double rotation)

The zig-zig order (grandparent first) is what makes splay trees efficient: it roughly **halves the depth** of every node on the access path.

```

SplayTree.java splay

// Brings 'key' (or the last node on its search path) to the root.
static Node splay(Node root, int key) {
    if (root == null || root.key == key) return root;
    if (key < root.key) {
        if (root.left == null) return root;
        if (key < root.left.key) {           // zig-zig (left-left)
            root.left.left = splay(root.left.left, key);
            root = rotateRight(root);
        } else if (key > root.left.key) {    // zig-zag (left-right)
            root.left.right = splay(root.left.right, key);
            if (root.left.right != null) root.left = rotateLeft(root.left);
        }
        return root.left == null ? root : rotateRight(root); // final zig
    } else {
        if (root.right == null) return root;
        if (key > root.right.key) {          // zag-zag (right-right)
            root.right.right = splay(root.right.right, key);
            root = rotateLeft(root);
        } else if (key < root.right.key) {   // zag-zig (right-left)
            root.right.left = splay(root.right.left, key);
            if (root.right.left != null) root.right = rotateRight(root.right);
        }
        return root.right == null ? root : rotateLeft(root); // final zag
    }
}

```

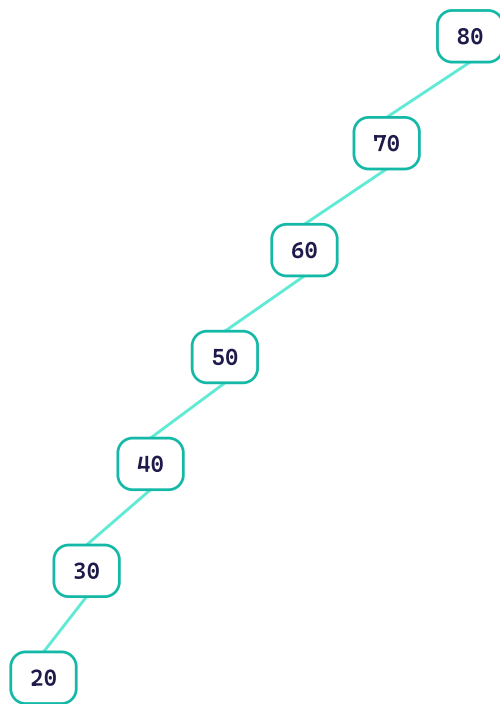
```

SplayTree.java insert

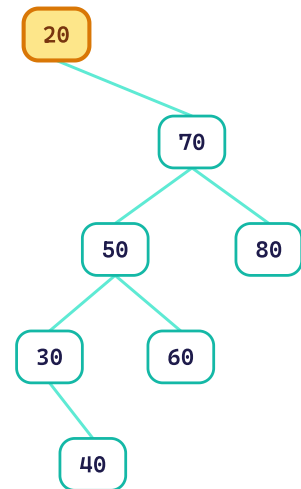
static Node insert(Node root, int key) {
    if (root == null) return new Node(key);
    root = splay(root, key);           // closest key is now the root
    if (root.key == key) return root;
    Node fresh = new Node(key);
    if (key < root.key) {               // split around the old root
        fresh.right = root;
        fresh.left = root.left;
        root.left = null;
    } else {
        fresh.left = root;
        fresh.right = root.right;
        root.right = null;
    }
    return fresh;                      // the new key becomes the root
}

```

Insertion splays the closest key to the root and then splits the tree around it, making the new key the root.



Before search(20)



After splay(20): depth roughly halved

 splayed to root

>_ OUTPUT OF SPLAYTREE.JAVA (ACTUAL RUN)

```
after inserts : 80(70(60(50(40(30(20,),),),),),)
search 20      : 20(,70(50(30(,40),60),80))
search 60      : 60(20(,50(30(,40),),),70(,80))
search 60 again (already root): 60(20(,50(30(,40),),),70(,80))
```

The output shows the weakness and the strength. Inserting keys can leave a long chain (after these inserts, 20 is at depth 6), so a **single** operation can cost $O(n)$. But splaying 20 pulls it to the root and roughly halves the depth of the path. The theory guarantees **$O(\log n)$ amortized** per operation: any sequence of m operations costs $O(m \log n)$ in total. A second access to the same key is $O(1)$.

8.2 Treaps

A **treap** (tree + heap) gives every node a **random priority** in addition to its key. The tree is a **BST by key** and a **heap by priority** (every parent's priority is higher than its children's). Insert as in a BST, then rotate the new node **up** while its priority beats its parent's. Because priorities are random, the tree has the shape of a BST built from a **random insertion order**, which has expected height $O(\log n)$ regardless of the actual order of the keys. Treaps are short to implement and support fast **split** and **merge**, which makes them popular in competitive programming.

8.3 Other balanced search trees

Tree	Idea	Guarantee
2-3 tree	nodes hold 1 or 2 keys; all leaves at the same depth; grows at the root by splitting	$O(\log n)$ worst case; red-black trees encode it
2-3-4 tree	nodes hold 1 to 3 keys; a B-tree of order 4	$O(\log n)$; equivalent to classic red-black trees
Scapegoat tree	no per-node data; when a node gets too deep, rebuild the smallest unbalanced ancestor subtree	$O(\log n)$ amortized
Weight-balanced tree	balance by subtree sizes instead of heights	$O(\log n)$ worst case
Skip list (not a tree)	layered linked lists with random “express lanes”	$O(\log n)$ expected; used in Redis and <code>ConcurrentSkipListMap</code>

Σ CHAPTER SUMMARY

- **Splay trees:** move every accessed node to the root with zig, zig-zig and zig-zag steps; $O(\log n)$ amortized, great for locality.
- **Treaps:** a BST by key and a heap by random priority; expected $O(\log n)$.
- 2-3 and 2-3-4 trees are the multi-key trees that red-black trees simulate, and they lead directly to B-trees.

CHAPTER 9

B-Trees and B+ Trees



9.1 Why binary trees are wrong for disks

A balanced BST with a billion keys has height about 30. In memory, 30 steps are nothing. But if the tree lives on disk, each step to a new node can mean a separate disk read, and one disk read costs as much as millions of memory operations. 30 disk reads per lookup is far too slow.

The fix: make each node **hold many keys**, so that one node fills an entire disk page (typically 4-16 KB). A node with hundreds of keys has hundreds of children, and the tree becomes extremely shallow. With 1,000 keys per node, **three levels hold a billion keys**. The root is usually cached in memory, so a lookup costs two or three disk reads.

9.2 The B-tree rules

A **B-tree of minimum degree t** ($t \geq 2$) satisfies:

1. Every node holds **at most $2t - 1$ keys**, and every node except the root holds **at least $t - 1$ keys**.
2. The keys in a node are **sorted**. An internal node with k keys has exactly **$k + 1$ children**; the keys act as separators: everything in child i lies between key $i - 1$ and key i .
3. **All leaves are at the same depth**. The tree is always perfectly balanced.
4. The tree grows and shrinks **at the root**, not at the leaves.

With $t = 2$, nodes hold 1 to 3 keys and have 2 to 4 children: that is the **2-3-4 tree**. Real databases use t in the hundreds. The height of a B-tree with n keys is at most **$\log_t((n + 1) / 2)$** .

```

BTree.java  node

static final int T = 2;           // minimum degree: a node holds T-1 .. 2T-1 keys

static class Node {
    int n;                        // keys currently stored
    int[] keys = new int[2 * T - 1]; // sorted keys
    Node[] children = new Node[2 * T]; // n + 1 children (internal nodes only)
    boolean leaf = true;
}

static Node root = new Node();
  
```

9.3 Search

Search inside the node for the first key $\geq$ the target (linear scan here; binary search for large nodes), then either stop or descend into the child between the keys:

```

BTree.java  search

static boolean search(Node x, int k) {
    int i = 0;
    while (i < x.n && k > x.keys[i]) i++;           // first key >= k
    if (i < x.n && k == x.keys[i]) return true;
    if (x.leaf) return false;
    return search(x.children[i], k);               // descend between keys
}

```

Cost: $O(\log_t n)$ nodes visited, $O(t)$ work per node: **$O(t \cdot \log_t n)$** CPU time, but only **$O(\log_t n)$ disk reads**, which is what matters.

9.4 Insert with proactive splitting

New keys always go into a **leaf**. If the leaf is full, it must be **split**: its median key moves up into the parent, and the remaining keys form two half-full nodes. If the parent is also full, it splits too, and so on up. If the root splits, a new root is created, and **that** is the only way the tree gets taller, which keeps all leaves at the same depth.



The CLRS algorithm splits **proactively**: while descending, any full node on the path is split **before** entering it. Then the parent always has room for a median key, and insertion never has to travel back up:

```

BTree.java  insert

static void insert(int k) {
    Node r = root;
    if (r.n == 2 * T - 1) {                          // full root: split it first,
                                                         // the tree grows taller HERE
        Node s = new Node();
        s.leaf = false;
        s.children[0] = r;
        root = s;
        splitChild(s, 0);
        insertNonFull(s, k);
    } else {
        insertNonFull(r, k);
    }
}

// x.children[i] is full: move its median key up into x and split it in two.
static void splitChild(Node x, int i) {
    Node y = x.children[i];
    Node z = new Node();
    z.leaf = y.leaf;
}

```

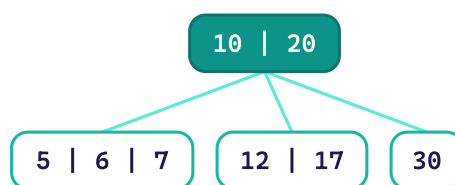
```

    z.n = T - 1;
    for (int j = 0; j < T - 1; j++) z.keys[j] = y.keys[j + T];           // upper keys
    if (!y.leaf) {
        for (int j = 0; j < T; j++) z.children[j] = y.children[j + T]; // upper children
    }
    y.n = T - 1;                                                         // y keeps lower
keys
    for (int j = x.n; j > i; j--) x.children[j + 1] = x.children[j];      // make room
    x.children[i + 1] = z;
    for (int j = x.n - 1; j >= i; j--) x.keys[j + 1] = x.keys[j];
    x.keys[i] = y.keys[T - 1];                                           // median goes
up
    x.n++;
}

static void insertNonFull(Node x, int k) {
    int i = x.n - 1;
    if (x.leaf) {
        while (i >= 0 && k < x.keys[i]) { // shift bigger keys right
            x.keys[i + 1] = x.keys[i];
            i--;
        }
        x.keys[i + 1] = k;
        x.n++;
    } else {
        while (i >= 0 && k < x.keys[i]) i--;
        i++; // child that should get k
        if (x.children[i].n == 2 * T - 1) { // split BEFORE descending
            splitChild(x, i);
            if (k > x.keys[i]) i++;
        }
        insertNonFull(x.children[i], k);
    }
}
}

```

`splitChild(x, i)` moves the upper $t - 1$ keys (and t children) of the full child into a new node `z`, keeps the lower $t - 1$ in `y`, and lifts the median into `x` between them.



B-tree ($t = 2$) after inserting 10, 20, 5, 6, 12, 30, 7, 17

■ root

>_ OUTPUT OF BTREE.JAVA (ACTUAL RUN)

```

insert 10:
  level 0: [10]
insert 20:
  level 0: [10 20]
insert 5:
  level 0: [5 10 20]
insert 6:
  level 0: [10]
  level 1: [5 6] [20]
insert 12:
  level 0: [10]
  level 1: [5 6] [12 20]
insert 30:
  level 0: [10]
  level 1: [5 6] [12 20 30]
insert 7:
  level 0: [10]
  level 1: [5 6 7] [12 20 30]
insert 17:
  level 0: [10 20]
  level 1: [5 6 7] [12 17] [30]
inorder: [5, 6, 7, 10, 12, 17, 20, 30]
search 6: true, search 15: false

```

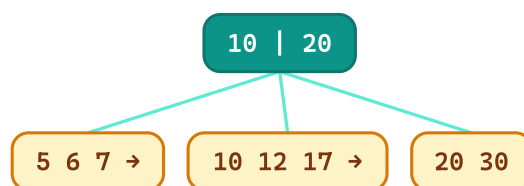
Follow the output: the root fills up to three keys [5 10 20]; inserting 6 splits it (10 moves up, the tree grows to two levels); later, inserting 17 finds the child [12 20 30] full, so it splits and 20 moves up into the root.

9.5 Deletion (overview)

Deletion must keep every non-root node at least $t - 1$ keys. Following CLRS, the algorithm again works **proactively** while descending: before entering a child that has only $t - 1$ keys, it makes sure the child has at least t keys by either **borrowing** a key from a sibling (through the parent, a rotation) or **merging** the child with a sibling and pulling a separator key down from the parent. To delete a key from an internal node, replace it with its predecessor or successor (from a leaf), just like BST deletion. If the root ends up empty, its only child becomes the new root, and the tree gets shorter. Every step is $O(t)$, with $O(\log_t n)$ levels.

9.6 B+ trees

Almost every real database index and file system uses a variant called the **B+ tree**:



B+ tree: internal nodes only route; all keys live in linked leaves

■ index node ■ leaf (data + next pointer)

	B-tree	B+ tree
Where data lives	in every node	only in the leaves
Internal nodes	keys + data	keys only (copies used for routing) → more keys per page, shallower tree
Leaves	not linked	linked left to right
Range scan	tree traversal	find the start leaf, then follow the leaf links: very fast
Search always ends at	any level	a leaf (predictable cost)
Used by	some file systems	MySQL InnoDB, PostgreSQL, SQLite, NTFS, ext4 (HTree), most databases

A query like `SELECT * FROM orders WHERE date BETWEEN '2026-01-01' AND '2026-01-31'` descends the B+ tree once to the first matching leaf, then reads leaves sequentially along the links.

Operation	B-tree / B+ tree (min degree t)
search	$O(\log_t n)$ disk reads, $O(t \log_t n)$ CPU
insert / delete	$O(\log_t n)$ disk reads, $O(t \log_t n)$ CPU
range query (k results)	$O(\log_t n + k/t)$ disk reads (B+ tree)
height	$\leq \log_t((n + 1) / 2)$

Σ CHAPTER SUMMARY

- B-trees store up to $2t - 1$ sorted keys per node, so the tree is **very shallow**: few disk reads per operation.
- All leaves are at the same depth; the tree grows by **splitting the root**.
- Insert splits full nodes on the way down (median moves up); delete borrows or merges on the way down.
- **B+ trees** keep data only in linked leaves: shallower trees and fast range scans. They power database indexes.

P A R T

III

Special-Purpose Trees

We build a MinHeap step by step, then tries, segment and Fenwick trees, N-ary trees, real applications, the Java collections, and a complete cheat sheet.

- 10 Build a Heap (Priority Queue)
- 11 Tries (Prefix Trees)
- 12 Segment Trees and Fenwick Trees
- 13 N-ary Trees and Tree Applications
- 14 Trees in the Java Collections Framework
- 15 Cheat Sheet: Choosing the Right Tree

CHAPTER 10

Build a Heap (Priority Queue)

10

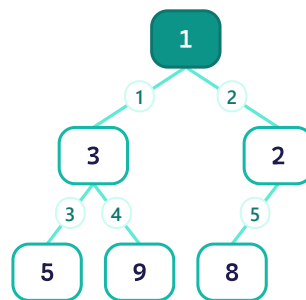
10.1 What a heap is

A **binary heap** is a **complete** binary tree with the **heap property**:

- **Min-heap**: every parent is $\leq$ its children, so the **minimum is always at the root**.
- **Max-heap**: every parent is $\geq$ its children, so the **maximum is at the root**.

A heap is **not** a BST: there's no order between left and right. It only guarantees that the extreme element is on top, and that is exactly what a **priority queue** needs ("give me the most urgent task next").

Because the tree is **complete**, we don't need `Node` objects at all: store it in an **array** in level order, using the index formulas from Chapter 2. Children of i are at $2i + 1$ and $2i + 2$; the parent is at $(i - 1) / 2$. The height is always $\lfloor \log_2 n \rfloor$.



Min-heap $[1, 3, 2, 5, 9, 8]$ drawn as a tree (index shown on each edge)

minimum

We'll build `MinHeap.java` in six steps.

10.2 Step 1: The skeleton (array, size, index helpers)

```
MinHeap.java skeleton

private int[] heap;           // heap[0..size-1] is a complete tree in level order
private int size;

public MinHeap(int capacity) {
    heap = new int[Math.max(1, capacity)];
}

public boolean isEmpty() {
    return size == 0;
}

public int size() {
    return size;
}
```

```

}

private static int parent(int i) { return (i - 1) / 2; }
private static int left(int i)   { return 2 * i + 1; }
private static int right(int i)  { return 2 * i + 2; }

private void swap(int a, int b) {
    int t = heap[a];
    heap[a] = heap[b];
    heap[b] = t;
}

private void ensureCapacity() { // full? double the array, like ArrayList
    if (size == heap.length) heap = Arrays.copyOf(heap, 2 * heap.length);
}

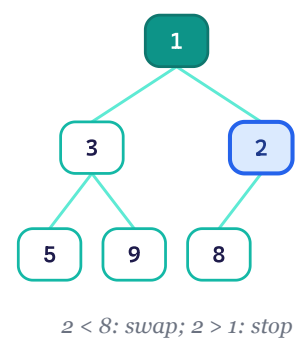
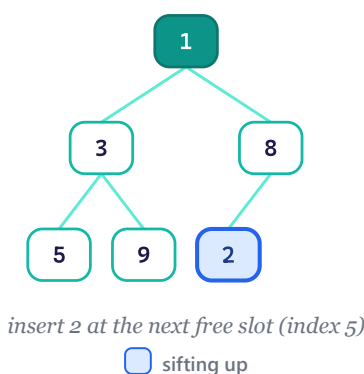
@Override
public String toString() {
    return Arrays.toString(Arrays.copyOf(heap, size));
}

```

- `heap[0..size-1]` holds the elements; slots beyond `size` are unused space.
- `parent`, `left`, `right` turn “tree navigation” into arithmetic. This is the whole trick of the array representation.
- `ensureCapacity` doubles the array when it’s full, like `ArrayList` does, so inserts stay **O(1) amortized** for the array part.

10.3 Step 2: insert with sift-up

Goal: add a value while keeping both properties. To stay **complete**, the new value must go into the **next free slot**, `heap[size]`. It might be smaller than its parent, breaking the **order**, so we **sift it up**: swap it with its parent while it is smaller.



? TRY IT FIRST

Write `siftUp(int i)`: while `i` isn’t the root and `heap[parent(i)] > heap[i]`, swap them and move `i` to the parent.

```

MinHeap.java  insert

public void insert(int value) {
    ensureCapacity();
    heap[size] = value;           // 1. next free slot keeps the tree complete
    siftUp(size);                 // 2. repair the order along the path to the root
    size++;
}

private void siftUp(int i) {
    while (i > 0 && heap[parent(i)] > heap[i]) { // smaller than parent: move up
        swap(i, parent(i));
        i = parent(i);
    }
}

```

At most one swap per level: **$O(\log n)$** . Only the path from the new slot to the root is touched; everything else is already in order.

10.4 Step 3: peek

```

MinHeap.java  peek

public int peek() {
    if (size == 0) throw new NoSuchElementException("heap is empty");
    return heap[0];           // the minimum is always the root
}

```

The minimum is always `heap[0]`: **$O(1)$** . This is the heap's superpower.

Checkpoint 1. The heap starts with capacity 4 and grows automatically:

```

MinHeap.java  checkInsert

MinHeap h = new MinHeap(4);           // small on purpose: it will grow
for (int x : new int[]{5, 3, 8, 1, 9, 2}) {
    h.insert(x);
    System.out.println("insert " + x + " -> " + h + "    min = " + h.peek());
}

```

>_ CHECKPOINT OUTPUT: MINHEAP.JAVA, INSERT (ACTUAL RUN)

```

insert 5 -> [5]    min = 5
insert 3 -> [3, 5]  min = 3
insert 8 -> [3, 5, 8]  min = 3
insert 1 -> [1, 3, 8, 5]  min = 1
insert 9 -> [1, 3, 8, 5, 9]  min = 1
insert 2 -> [1, 3, 2, 5, 9, 8]  min = 1

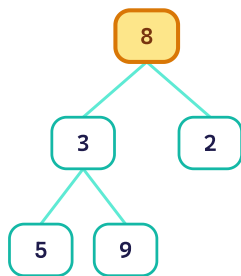
```

Look at the insertion of 2: it went to index 5 (under 8), then sifted up past 8, which is exactly the diagrams above.

10.5 Step 4: poll with sift-down

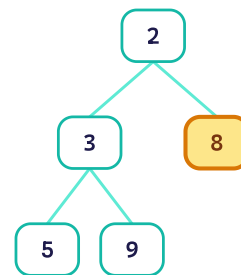
Goal: remove and return the minimum. We can't just delete `heap[0]` (that would leave a hole at the top). Instead:

1. Move the **last** element into the root (the tree stays complete).
2. It is probably too big for the root, so **sift it down**: swap it with its **smaller** child until it is smaller than both children.



poll(): last element 8 moves to the root

 sifting down



swap with the smaller child 2; now in place

? TRY IT FIRST

Why must we swap with the *smaller* child? (Try swapping 8 with 3 instead of 2 in the picture: is $3 \leq 2$ afterwards?)

```

MinHeap.java poll

public int poll() {
    // remove and return the minimum
    int min = peek();
    size--;
    heap[0] = heap[size];
    siftDown(0);
    return min;
}

private void siftDown(int i) {
    while (true) {
        int smallest = i;
        if (left(i) < size && heap[left(i)] < heap[smallest]) smallest = left(i);
        if (right(i) < size && heap[right(i)] < heap[smallest]) smallest = right(i);
        if (smallest == i) return; // smaller than both children: in place
        swap(i, smallest);        // swap with the SMALLER child
        i = smallest;
    }
}
  
```

`siftDown` finds the smallest of the node and its (up to two) children. If the node itself is smallest, we're done; otherwise swap and continue from the child's position. The bounds checks `left(i) < size` handle nodes with 0 or 1 children. **$O(\log n)$** .

Checkpoint 2. Polling repeatedly returns the values in sorted order:

MinHeap.java checkPoll

```
List<Integer> out = new ArrayList<>();
while (!h.isEmpty()) {
    out.add(h.poll());
    System.out.println("poll -> " + out.get(out.size() - 1) + "    heap now " + h);
}
System.out.println("order of polls: " + out);
```

>_ CHECKPOINT OUTPUT: MINHEAP.JAVA, POLL (ACTUAL RUN)

```
poll -> 1    heap now [2, 3, 8, 5, 9]
poll -> 2    heap now [3, 5, 8, 9]
poll -> 3    heap now [5, 9, 8]
poll -> 5    heap now [8, 9]
poll -> 8    heap now [9]
poll -> 9    heap now []
order of polls: [1, 2, 3, 5, 8, 9]
```

10.6 Step 5: fromArray, building a heap in $O(n)$

Goal: turn an existing array into a heap. Inserting n values one by one costs $O(n \log n)$. **Heapify** is faster: copy the array as is, then sift down every internal node, **from the last internal node (index $n/2 - 1$) back to the root**. Leaves are already valid one-node heaps.

MinHeap.java buildHeap

```
// O(n): copy the array, then sift down every internal node from the bottom up.
public static MinHeap fromArray(int[] values) {
    MinHeap h = new MinHeap(values.length);
    h.heap = Arrays.copyOf(values, Math.max(1, values.length));
    h.size = values.length;
    for (int i = values.length / 2 - 1; i >= 0; i--) h.siftDown(i); // leaves are already
    heaps
    return h;
}
```

Why $O(n)$? Half the nodes are leaves (0 work), a quarter sift down at most 1 level, an eighth at most 2 levels, and so on. The sum $n/4 \cdot 1 + n/8 \cdot 2 + n/16 \cdot 3 + \dots$ is less than n . Most nodes are near the bottom, where sifting down is cheap.

Checkpoint 3.

MinHeap.java checkBuild

```
MinHeap b = MinHeap.fromArray(new int[]{9, 4, 7, 1, 8, 2, 6, 3});
System.out.println("fromArray -> " + b + " valid heap? " + b.isValidHeap());
```

>_ CHECKPOINT OUTPUT: MINHEAP.JAVA, BUILD (ACTUAL RUN)

```
fromArray -> [1, 3, 2, 4, 8, 7, 6, 9] valid heap? true
```

10.7 Step 6: heap sort

Goal: sort an array **in place** in $O(n \log n)$ worst case. Use a **max-heap**: build it in $O(n)$, then repeatedly swap the maximum (the root) with the last element of the heap, shrink the heap by one, and sift the new root down.

MinHeap.java heapSort

```
public static void heapSort(int[] a) { // ascending, in place
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) maxSiftDown(a, n, i); // build a MAX-heap
    for (int end = n - 1; end > 0; end--) {
        int t = a[0]; a[0] = a[end]; a[end] = t; // largest goes to the end
        maxSiftDown(a, end, 0); // heap shrinks by one
    }
}

private static void maxSiftDown(int[] a, int n, int i) {
    while (true) {
        int largest = i, l = 2 * i + 1, r = l + 1;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) return;
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        i = largest;
    }
}
```

The sorted part grows from the right end of the array while the heap shrinks on the left. Heap sort uses $O(1)$ extra memory and is always $O(n \log n)$, but it isn't stable and is cache-unfriendly, so quicksort/merge sort are usually faster in practice.

Checkpoint 4.

MinHeap.java checkSort

```
int[] a = {12, 11, 13, 5, 6, 7};
MinHeap.heapSort(a);
System.out.println("heapSort -> " + Arrays.toString(a));
```

>_ CHECKPOINT OUTPUT: MINHEAP.JAVA, SORT (ACTUAL RUN)

```
heapSort -> [5, 6, 7, 11, 12, 13]
```

10.8 The same thing in Java: PriorityQueue

`java.util.PriorityQueue` is exactly the class we just wrote (a min-heap in a growable array), generalized to any type with a `Comparator`. For a max-heap, pass `Collections.reverseOrder()`.

A classic pattern: to find the **k largest** elements, keep a **min-heap of size k**. Every new value goes in; if the heap grows past k, poll the smallest. The heap then holds the k largest values, and its root is the k-th largest: **$O(n \log k)$** time, **$O(k)$** memory.

```
MinHeap.java pq

static int kthLargest(int[] nums, int k) {           // min-heap holding the k largest
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int x : nums) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();               // evict the smallest
    }
    return pq.peek();
}
```

```
MinHeap.java checkPq

PriorityQueue<Integer> minPq = new PriorityQueue<>(List.of(5, 1, 4));
PriorityQueue<Integer> maxPq = new PriorityQueue<>(Collections.reverseOrder());
maxPq.addAll(List.of(5, 1, 4));
System.out.println("java min-heap peek=" + minPq.peek() + " max-heap peek=" + maxPq.peek());
System.out.println("2nd largest of [3,2,1,5,6,4] = " + kthLargest(new int[]{3, 2, 1, 5, 6, 4}, 2));
```

>_ CHECKPOINT OUTPUT: MINHEAP.JAVA, PQ (ACTUAL RUN)

```
java min-heap peek=1 max-heap peek=5
2nd largest of [3,2,1,5,6,4] = 5
```

10.9 Your finished MinHeap

Method	What it does	Time
<code>peek()</code>	read the minimum	$O(1)$
<code>insert(v)</code>	append + sift up	$O(\log n)$ (amortized, including growth)
<code>poll()</code>	move last to root + sift down	$O(\log n)$
<code>fromArray(a)</code>	heapify	$O(n)$
<code>heapSort(a)</code>	build max-heap + n polls	$O(n \log n)$, in place
<code>size</code> , <code>isEmpty</code>	counters	$O(1)$
search for an arbitrary value	not supported efficiently	$O(n)$

Other heaps exist for special needs: **d-ary heaps** (shallower, faster decrease-key), **binomial** and **Fibonacci heaps** (fast merging; Fibonacci heaps give $O(1)$ amortized decrease-key, which speeds up Dijkstra's algorithm in theory).

Σ CHAPTER SUMMARY

- A heap is a **complete** tree stored in an **array**; each parent beats its children.
- `insert` = next free slot + **sift up**; `poll` = last element to root + **sift down** (swap with the smaller child). Both $O(\log n)$; `peek` $O(1)$.
- Heapify from the last internal node down to the root is **$O(n)$** ; heap sort is $O(n \log n)$ in place.
- Java's `PriorityQueue` is the same structure. Top-k: min-heap of size k .

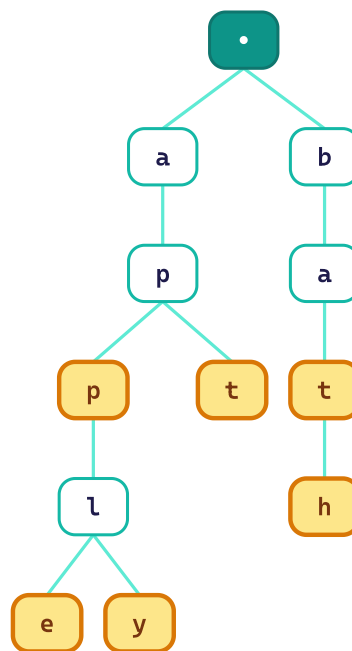
CHAPTER 11

Tries (Prefix Trees)

11

11.1 Keys spelled out along paths

A **trie** (from *retrieval*, usually pronounced “try”) stores strings by their **characters**: each edge is a letter, and each path from the root spells a prefix. Words that share a prefix share a path. A flag in the node marks where a complete word ends.



Trie holding app, apple, apply, apt, bat, bath (highlighted = end of a word)

 isWord = true

Operations depend on the **length of the word** (L), not on how many words are stored. Looking up “apply” takes 5 steps whether the trie holds 10 words or 10 million.

```

Trie.java  node

static class Node {
    Node[] next = new Node[26]; // one slot per letter 'a'..'z'
    boolean isWord;             // a word ends exactly here
    int prefixCount;            // how many words pass through this node
}

private final Node root = new Node();
  
```

Each node has an array of 26 children (for lowercase English letters). `prefixCount` counts how many words pass through the node, which answers “how many words start with this prefix?” instantly.

11.2 Insert, search and prefix queries

```
Trie.java insert

void insert(String word) {
    Node cur = root;
    for (char c : word.toCharArray()) {
        int i = c - 'a';
        if (cur.next[i] == null) cur.next[i] = new Node(); // create the path lazily
        cur = cur.next[i];
        cur.prefixCount++;
    }
    cur.isWord = true;
}
```

```
Trie.java search

private Node walk(String s) { // follow s; null if the path breaks
    Node cur = root;
    for (char c : s.toCharArray()) {
        cur = cur.next[c - 'a'];
        if (cur == null) return null;
    }
    return cur;
}

boolean search(String word) {
    Node end = walk(word);
    return end != null && end.isWord; // path exists AND a word ends there
}

boolean startsWith(String prefix) {
    return walk(prefix) != null;
}

int countWithPrefix(String prefix) {
    Node end = walk(prefix);
    return end == null ? 0 : end.prefixCount;
}
```

`search` and `startsWith` walk the same path. The difference: `search("appl")` fails because no word **ends** at that node (`isWord` is false), while `startsWith("appl")` succeeds because the **path** exists.

11.3 Autocomplete

Walk to the prefix's node, then collect every word below it with a depth-first traversal. Visiting children in index order returns the words **alphabetically**.

Trie.java autocomplete

```

List<String> autocomplete(String prefix) {
    List<String> out = new ArrayList<>();
    Node start = walk(prefix);
    if (start != null) collect(start, new StringBuilder(prefix), out);
    return out;
}

private void collect(Node node, StringBuilder sb, List<String> out) {
    if (node.isWord) out.add(sb.toString());
    for (int i = 0; i < 26; i++) {           // alphabetical order for free
        if (node.next[i] == null) continue;
        sb.append((char) ('a' + i));
        collect(node.next[i], sb, out);
        sb.deleteCharAt(sb.length() - 1);
    }
}

```

11.4 Delete

Unmark the word's end, then remove nodes that no longer lead to any word, walking back up the recursion:

Trie.java delete

```

void delete(String word) {
    if (search(word)) delete(root, word, 0);
}

// Returns true if 'node' is no longer needed and can be unlinked by its parent.
private boolean delete(Node node, String word, int depth) {
    if (depth == word.length()) {
        node.isWord = false;
    } else {
        int i = word.charAt(depth) - 'a';
        Node child = node.next[i];
        child.prefixCount--;
        if (delete(child, word, depth + 1)) node.next[i] = null; // prune empty branch
    }
    return node != root && !node.isWord && node.prefixCount == 0;
}

```

>_ OUTPUT OF TRIE.JAVA (ACTUAL RUN)

```

search(app)=true  search(appl)=false  startsWith(appl)=true
words with prefix 'ap': 4, with 'bat': 3
autocomplete(ap)  = [app, apple, apply, apt]
autocomplete(bat) = [bat, bath, batman]
after delete(apple): search(apple)=false search(apply)=true autocomplete(ap)=[app, apply, apt]

```

Operation	Trie	HashSet of strings	Balanced BST of strings
insert / search / delete	$O(L)$	$O(L)$ average (hashing)	$O(L \log n)$
prefix search / count	$O(L)$	$O(n \cdot L)$	$O(L \log n + k)$
autocomplete (k results)	$O(L + \text{size of subtree})$	$O(n \cdot L)$	$O(L \log n + k)$
sorted iteration	yes	no	yes
memory	high (26 references per node)	low	medium

✓ SAVING MEMORY

With 26 references per node, tries can use a lot of memory. Common fixes: use a `HashMap<Character, Node>` for children (good for large alphabets like Unicode); or a **compressed trie** (radix tree, Patricia trie), which merges chains of single-child nodes into one edge labeled with a whole substring. Radix trees are used in IP routing tables and in many databases' in-memory indexes.

Σ CHAPTER SUMMARY

- A trie stores strings along paths; shared prefixes share nodes; `isWord` marks word ends.
- Insert, search, `startsWith`: **$O(L)$** , independent of the number of words.
- Ideal for **prefix queries and autocomplete**; memory is the main cost (compressed tries help).

CHAPTER 12

Segment Trees and Fenwick Trees

12

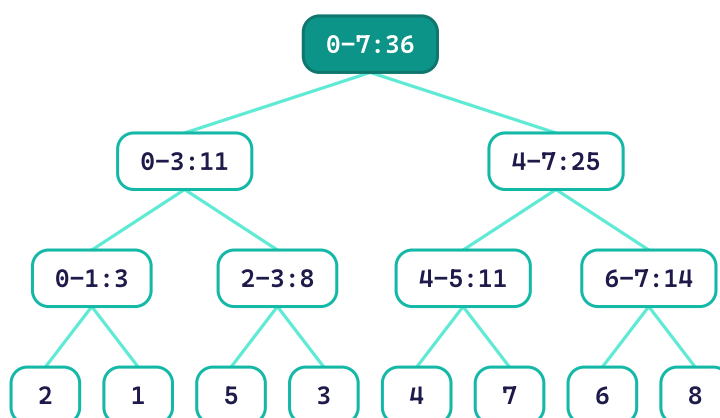
12.1 The problem: range queries with updates

Given an array, answer many queries like “sum of elements from index l to r ” while elements also change. The two naive approaches are each slow for one of the operations:

Approach	Query $\text{sum}(l, r)$	Update $a[i]$
loop over the range	$O(n)$	$O(1)$
prefix-sum array	$O(1)$	$O(n)$ (rebuild prefixes)
segment tree / Fenwick tree	$O(\log n)$	$O(\log n)$

12.2 Segment tree

A **segment tree** is a binary tree where each node stores an answer (here, a sum) for a **range** of the array. The root covers the whole array; each node’s children cover the left and right halves of its range; the leaves are single elements.



Segment tree over $[2, 1, 5, 3, 4, 7, 6, 8]$: each node stores the sum of its range

whole array

It is stored in an array like a heap: node i has children $2i$ and $2i + 1$ (1-based). An array of size $4n$ is always enough.

```

RangeTrees.java segmentTree

static class SegmentTree {
    private final int n;
    private final long[] tree;           // tree[node] = sum of the segment it covers

    SegmentTree(int[] a) {
        n = a.length;
    }
}

```

```

    tree = new long[4 * n];          // 4n is always enough room
    build(a, 1, 0, n - 1);
}

private void build(int[] a, int node, int lo, int hi) {
    if (lo == hi) {                 // leaf: a single element
        tree[node] = a[lo];
        return;
    }
    int mid = (lo + hi) / 2;
    build(a, 2 * node, lo, mid);     // left child covers [lo, mid]
    build(a, 2 * node + 1, mid + 1, hi); // right child covers [mid+1, hi]
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

long query(int l, int r) {
    return query(1, 0, n - 1, l, r);
}

private long query(int node, int lo, int hi, int l, int r) {
    if (r < lo || hi < l) return 0; // no overlap
    if (l <= lo && hi <= r) return tree[node]; // total overlap: use stored sum
    int mid = (lo + hi) / 2;        // partial overlap: ask children
    return query(2 * node, lo, mid, l, r) + query(2 * node + 1, mid + 1, hi, l, r);
}

void update(int index, int value) {
    update(1, 0, n - 1, index, value);
}

private void update(int node, int lo, int hi, int index, int value) {
    if (lo == hi) {
        tree[node] = value;
        return;
    }
    int mid = (lo + hi) / 2;
    if (index <= mid) update(2 * node, lo, mid, index, value);
    else update(2 * node + 1, mid + 1, hi, index, value);
    tree[node] = tree[2 * node] + tree[2 * node + 1]; // recompute on the way up
}
}

```

- **Build** fills the tree bottom-up: $O(n)$.
- **Query** splits into three cases per node: no overlap with $[l, r]$ (contributes 0), total overlap (use the stored sum, no further descent), partial overlap (ask both children). At most about 4 nodes per level are visited: $O(\log n)$.
- **Update** walks from the root to one leaf, then recomputes the sums on the way back up: $O(\log n)$.

Replace `+` with `min`, `max`, `gcd` or any other **associative** operation, and the same code answers range-minimum, range-maximum, or range-gcd queries. With **lazy propagation** (storing pending updates at internal nodes and pushing them down only when needed), a segment tree can also update a whole **range** (for example, “add 5 to every element in $[l, r]$ ”) in $O(\log n)$.

12.3 Fenwick tree (binary indexed tree)

A **Fenwick tree** answers prefix sums and point updates in $O(\log n)$ with a much smaller structure: an array of $n + 1$ numbers and two tiny loops. Index i stores the sum of a block of elements ending at i , whose length is $i \& -i$, the lowest set bit of i .

```
RangeTrees.java fenwick

static class FenwickTree {
    private final long[] bit;          // 1-based; bit[i] covers (i - lowbit(i), i]

    FenwickTree(int n) {
        bit = new long[n + 1];
    }

    void add(int i, long delta) {      // a[i] += delta (i is 0-based)
        for (i++; i < bit.length; i += i & -i) bit[i] += delta;
    }

    long prefixSum(int i) {             // a[0] + ... + a[i]
        long s = 0;
        for (i++; i > 0; i -= i & -i) s += bit[i];
        return s;
    }

    long rangeSum(int l, int r) {
        return prefixSum(r) - (l > 0 ? prefixSum(l - 1) : 0);
    }
}
```

- `prefixSum(i)` adds blocks while **removing** the lowest set bit: $i \rightarrow i - (i \& -i)$. For example, `prefix(13)` = block 13 (length 1) + block 12 (length 4) + block 8 (length 8).
- `add(i, delta)` updates every block containing i by **adding** the lowest set bit: $i \rightarrow i + (i \& -i)$.
- Both loops run at most $\log_2 n$ times.

>_ OUTPUT OF RANGETREES.JAVA (ACTUAL RUN)

```
array [2, 1, 5, 3, 4, 7, 6, 8]
sum[2..5]: segment tree = 19, fenwick = 19
sum[0..7]: segment tree = 36, fenwick = 36
after a[3] = 10 -> sum[2..5]: segment tree = 26, fenwick = 26
```

	Segment tree	Fenwick tree
Point update, range query	$O(\log n)$	$O(\log n)$ (sums, or any invertible operation)
Range update (lazy)	$O(\log n)$	only with tricks (two trees)
Operations supported	any associative (min, max, gcd, sum...)	invertible ones (sum, xor); prefix-min only in special cases
Memory	$4n$	$n + 1$
Code size	~40 lines	~10 lines

Σ CHAPTER SUMMARY

- Both structures give **$O(\log n)$** range queries **and** point updates.
- **Segment tree**: each node stores the answer for a range; supports any associative operation and, with lazy propagation, range updates.
- **Fenwick tree**: tiny and fast for prefix sums, using the lowest set bit `i & -i`.

CHAPTER 13

N-ary Trees and Tree Applications

13

13.1 N-ary (general) trees

When a node can have **any number of children**, store them in a list:

```
NaryTree.java  node

static class Node {
    String name;
    int size; // e.g. file size in KB (0 for folders)
    List<Node> children = new ArrayList<>();

    Node(String name, int size) {
        this.name = name;
        this.size = size;
    }

    Node add(Node child) {
        children.add(child);
        return this;
    }
}
```

The traversals generalize directly. Preorder visits a node, then each child; postorder visits each child, then the node. There is no inorder, because there is no single “middle” position. Level order is the same queue algorithm:

```

NaryTree.java traversals

static void preorder(Node n, List<String> out) {    // node, then each child
    out.add(n.name);
    for (Node c : n.children) preorder(c, out);
}

static void postorder(Node n, List<String> out) {    // each child, then node
    for (Node c : n.children) postorder(c, out);
    out.add(n.name);
}

static List<List<String>> levelOrder(Node root) {
    List<List<String>> levels = new ArrayList<>();
    Queue<Node> q = new ArrayDeque<>();
    q.add(root);
    while (!q.isEmpty()) {
        List<String> level = new ArrayList<>();
        for (int k = q.size(); k > 0; k--) {
            Node n = q.poll();
            level.add(n.name);
            q.addAll(n.children);
        }
        levels.add(level);
    }
    return levels;
}

```

Many real computations are **postorder**: a folder's total size is its own size plus the total sizes of its children, which you can only know after visiting the children (this is what the `du` command does).

```

NaryTree.java measures

static int height(Node n) {    // edges
    int h = -1;
    for (Node c : n.children) h = Math.max(h, height(c));
    return h + 1;
}

static int totalSize(Node n) {    // like "du": sum over the subtree
    int total = n.size;
    for (Node c : n.children) total += totalSize(c);
    return total;
}

```


 NaryTree.java print

```
static void print(Node n, String prefix, boolean last, StringBuilder sb) {
    sb.append(prefix).append(last ? "└─ " : "├─ ").append(n.name)
      .append(" (").append(totalSize(n)).append(" KB)\n");
    String childPrefix = prefix + (last ? "    " : "|    ");
    for (int i = 0; i < n.children.size(); i++) {
        print(n.children.get(i), childPrefix, i == n.children.size() - 1, sb);
    }
}
```

>_ OUTPUT OF NARYTREE.JAVA (ACTUAL RUN)

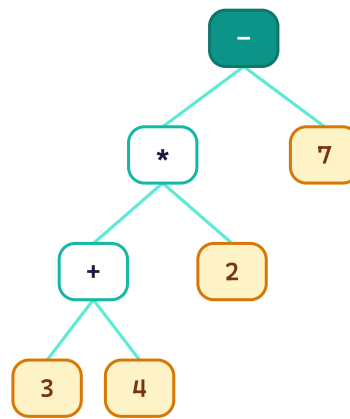
```
└─ project (964 KB)
  └─ src (60 KB)
    └─ Main.java (12 KB)
      └─ util (48 KB)
        └─ Tree.java (30 KB)
          └─ Heap.java (18 KB)
      └─ docs (900 KB)
        └─ guide.pdf (900 KB)
    └─ README.md (4 KB)
preorder : [project, src, Main.java, util, Tree.java, Heap.java, docs, guide.pdf, README.md]
postorder: [Main.java, Tree.java, Heap.java, util, src, guide.pdf, docs, README.md, project]
levels   : [[project], [src, docs, README.md], [Main.java, util, guide.pdf], [Tree.java, Heap.java]]
height   : 3
```

I LEFT-CHILD, RIGHT-SIBLING

Any N-ary tree can be stored as a **binary** tree: each node keeps a pointer to its **first child** and one to its **next sibling**. This uses exactly two references per node regardless of the number of children, and all binary tree algorithms can then be reused.

13.2 Expression trees

An arithmetic expression is naturally a tree: operators are internal nodes, numbers are leaves. Compilers build such trees (**abstract syntax trees**) from source code.

Expression tree for $((3 + 4) * 2) - 7$

■ operator ■ operand

```

TreeApplications.java  expression

static class Expr {
    String token;
    Expr left, right;           // operators have two children, numbers none

    Expr(String token, Expr left, Expr right) {
        this.token = token;
        this.left = left;
        this.right = right;
    }
}

static Expr fromPostfix(String postfix) {
    Deque<Expr> stack = new ArrayDeque<>();
    for (String tok : postfix.split(" ")) {
        if ("+-*/".contains(tok)) {
            Expr right = stack.pop();           // popped in reverse order
            Expr left = stack.pop();
            stack.push(new Expr(tok, left, right));
        } else {
            stack.push(new Expr(tok, null, null));
        }
    }
    return stack.pop();
}

static double evaluate(Expr e) {             // postorder: children first, then operator
    if (e.left == null) return Double.parseDouble(e.token);
    double a = evaluate(e.left), b = evaluate(e.right);
    return switch (e.token) {
        case "+" -> a + b;
        case "-" -> a - b;
        case "*" -> a * b;
        default -> a / b;
    };
}

```

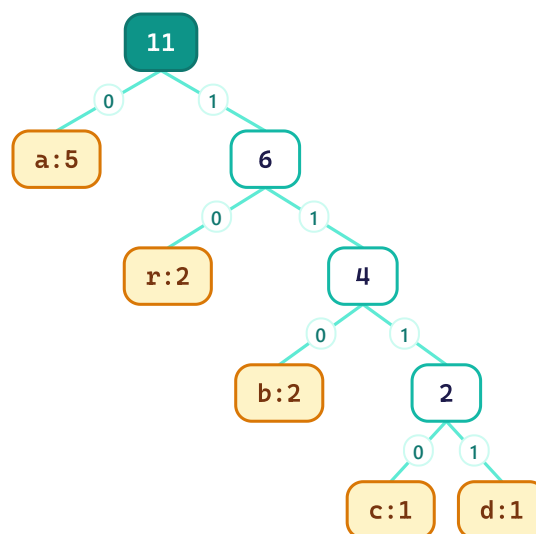
```
static String infix(Expr e) { // inorder with parentheses
    if (e.left == null) return e.token;
    return "(" + infix(e.left) + " " + e.token + " " + infix(e.right) + ")";
}
```

- **Building from postfix** uses a stack: push numbers; for an operator, pop the right operand, then the left, and push a new node.
- **Evaluation** is a **postorder** traversal: evaluate both children, then apply the operator.
- The three traversals produce the three notations: preorder gives **prefix** (`- * + 3 4 2 7`), inorder with parentheses gives **infix**, postorder gives **postfix** (`3 4 + 2 * 7 -`).

13.3 Huffman coding

Huffman coding is an optimal way to compress text by giving **frequent characters shorter codes**. It builds a full binary tree bottom-up with a priority queue:

1. Create a leaf for each character, weighted by its frequency.
2. Repeatedly remove the **two lightest** trees and merge them under a new node whose weight is their sum.
3. When one tree remains, each character's code is the path from the root: left = 0, right = 1.



Huffman tree for "abracadabra" (left edge = 0, right edge = 1)

 character : frequency

```
TreeApplications.java huffman

static class HNode {
    char ch;
    int freq;
    HNode left, right;

    HNode(char ch, int freq, HNode left, HNode right) {
        this.ch = ch;
        this.freq = freq;
        this.left = left;
    }
}
```

```

        this.right = right;
    }
}

static Map<Character, String> huffmanCodes(String text) {
    Map<Character, Integer> freq = new TreeMap<>();
    for (char c : text.toCharArray()) freq.merge(c, 1, Integer::sum);

    PriorityQueue<HNode> pq = new PriorityQueue<>((
        (a, b) -> a.freq != b.freq ? a.freq - b.freq : a.ch - b.ch); // deterministic ties
    for (var e : freq.entrySet()) pq.add(new HNode(e.getKey(), e.getValue(), null, null));

    while (pq.size() > 1) { // repeatedly merge the two rarest trees
        HNode a = pq.poll(), b = pq.poll();
        pq.add(new HNode((char) Math.min(a.ch, b.ch), a.freq + b.freq, a, b));
    }
    Map<Character, String> codes = new TreeMap<>();
    assign(pq.poll(), "", codes);
    return codes;
}

static void assign(HNode n, String code, Map<Character, String> codes) {
    if (n.left == null) { // leaf = a real character
        codes.put(n.ch, code.isEmpty() ? "0" : code);
        return;
    }
    assign(n.left, code + "0", codes); // left edge = 0
    assign(n.right, code + "1", codes); // right edge = 1
}

```

Because characters sit only at leaves, **no code is a prefix of another**, so the bit stream decodes without separators. “abracadabra” needs 23 bits instead of 88. The same idea is used inside ZIP (DEFLATE), JPEG and MP3. Building the tree takes $O(k \log k)$ for k distinct characters.

13.4 Union-find (disjoint set union)

A **union-find** structure tracks a collection of disjoint sets, supporting “are a and b in the same set?” and “merge the sets of a and b ”. It is a **forest**: each set is a tree, identified by its root, and each element stores only its parent.

```

TreeApplications.java unionFind

static class UnionFind {
    int[] parent, rank;

    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i; // everyone is their own root
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false; // already in the same set
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra; // attach shorter tree under taller
        if (rank[ra] == rank[rb]) rank[ra]++;
        return true;
    }
}

```

- `find` follows parents up to the root. **Path compression** points every visited node directly at the root, flattening the tree for next time.
- `union` links the root of the shorter tree under the root of the taller one (**union by rank**), keeping the trees shallow.

With both optimizations, each operation takes $O(\alpha(n))$ amortized, where α is the inverse Ackermann function: at most 4 for any input size that fits in the universe, so effectively constant. Union-find is the engine of **Kruskal's minimum spanning tree algorithm**, network connectivity, image segmentation and cycle detection.

>_ OUTPUT OF TREEAPPLICATIONS.JAVA (ACTUAL RUN)

```

postfix 3 4 + 2 * 7 - -> infix (((3 + 4) * 2) - 7) = 7.0
huffman codes for "abracadabra": {a=0, b=110, c=1110, d=1111, r=10}
encoded: 01101001110011110110100 (23 bits vs 88 bits in 8-bit ASCII)
union(0,1) merged
union(1,2) merged
union(3,4) merged
union(5,6) merged
union(6,7) merged
union(2,0) already connected
connected(0,2)=true connected(2,4)=false

```

CHAPTER SUMMARY

- N-ary trees store a list of children; traversals are preorder, postorder and level order.
- **Expression trees:** operators inside, operands at leaves; evaluate with postorder.
- **Huffman trees:** merge the two rarest trees repeatedly; codes are root-to-leaf paths.
- **Union-find:** a forest with path compression and union by rank; near-constant operations.

CHAPTER 14

Trees in the Java Collections Framework

14

You rarely need to implement a balanced tree yourself in Java. The standard library provides them, and knowing their exact capabilities saves a lot of work.

Class	Tree inside	Key operations and cost
<code>TreeMap<K, V></code>	red-black tree	<code>put</code> , <code>get</code> , <code>remove</code> , <code>containsKey</code> : $O(\log n)$; keys kept sorted
<code>TreeSet<E></code>	a <code>TreeMap</code> with dummy values	<code>add</code> , <code>remove</code> , <code>contains</code> : $O(\log n)$
<code>PriorityQueue<E></code>	binary heap in an array	<code>offer</code> , <code>poll</code> : $O(\log n)$; <code>peek</code> : $O(1)$
<code>HashMap</code> (large buckets)	red-black tree per crowded bucket	$O(\log n)$ worst case per bucket
<code>ConcurrentSkipListMap</code>	skip list	thread-safe sorted map, $O(\log n)$ expected

14.1 TreeMap: a sorted map with navigation

JavaTreeCollections.java treemap

```

TreeMap<String, Integer> scores = new TreeMap<>(); // red-black tree, sorted by key
scores.put("maya", 88);
scores.put("arjun", 72);
scores.put("zoe", 95);
scores.put("kiran", 64);
scores.put("dev", 81);
System.out.println("TreeMap (sorted) : " + scores);
System.out.println("firstKey / lastKey : " + scores.firstKey() + " / " + scores.lastKey());
System.out.println("floorKey(\"m\") : " + scores.floorKey("m")); // largest <= "m"
System.out.println("ceilingKey(\"m\") : " + scores.ceilingKey("m")); // smallest >= "m"
System.out.println("headMap(\"kiran\") : " + scores.headMap("kiran")); // keys < "kiran"
System.out.println("subMap(\"b\", \"n\") : " + scores.subMap("b", "n"));
System.out.println("descendingMap : " + scores.descendingMap());

```

Beyond a normal `Map`, `TreeMap` implements `NavigableMap`: `firstKey` / `lastKey`, `floorKey` ($\leq$), `ceilingKey` ($\geq$), `lowerKey` ($<$), `higherKey` ($>$), range views `headMap`, `tailMap`, `subMap` (views, not copies), `descendingMap`, and `pollFirstEntry` / `pollLastEntry`. Every navigation method is $O(\log n)$; a range view iterates in $O(\log n + k)$.

14.2 TreeSet: a sorted set

JavaTreeCollections.java `treeset`

```
TreeSet<Integer> set = new TreeSet<>(java.util.List.of(40, 10, 70, 20, 60, 30));
System.out.println("TreeSet          : " + set);
System.out.println("floor(35)=" + set.floor(35) + " ceiling(35)=" + set.ceiling(35)
    + " lower(40)=" + set.lower(40) + " higher(40)=" + set.higher(40));
System.out.println("subSet(20, 60)    : " + set.subSet(20, 60) + " (60 excluded)");
System.out.println("pollFirst()       : " + set.pollFirst() + " -> " + set);
```

14.3 PriorityQueue with a comparator

JavaTreeCollections.java `pq`

```
PriorityQueue<Task> tasks = new PriorityQueue<>(
    (a, b) -> Integer.compare(a.priority(), b.priority())); // smallest priority first
tasks.add(new Task("write report", 3));
tasks.add(new Task("fix prod bug", 1));
tasks.add(new Task("lunch", 2));
StringBuilder order = new StringBuilder();
while (!tasks.isEmpty()) order.append(tasks.poll().name()).append(" | ");
System.out.println("PriorityQueue order: " + order);
```

>_ OUTPUT OF JAVATREECOLLECTIONS.JAVA (ACTUAL RUN)

```
TreeMap (sorted)    : {arjun=72, dev=81, kiran=64, maya=88, zoe=95}
firstKey / lastKey  : arjun / zoe
floorKey("m")       : kiran
ceilingKey("m")      : maya
headMap("kiran")     : {arjun=72, dev=81}
subMap("b", "n")     : {dev=81, kiran=64, maya=88}
descendingMap       : {zoe=95, maya=88, kiran=64, dev=81, arjun=72}
TreeSet             : [10, 20, 30, 40, 60, 70]
floor(35)=30 ceiling(35)=40 lower(40)=30 higher(40)=60
subSet(20, 60)      : [20, 30, 40] (60 excluded)
pollFirst()         : 10 -> [20, 30, 40, 60, 70]
PriorityQueue order: fix prod bug | lunch | write report |
HashMap (no order guarantee): {zoe=95, dev=81, kiran=64, arjun=72, maya=88}
```

✗ COMMON MISTAKES

- **Iterating a PriorityQueue is not sorted.** Only repeated `poll()` returns elements in priority order; `toString()` and for-each show the internal array.
- **Keys must be comparable consistently.** A `TreeMap` uses `compareTo` (or the comparator), not `equals`. Two keys that compare as 0 are treated as the same key.
- **Don't mutate keys** inside a `TreeMap` / `TreeSet` (or elements inside a `PriorityQueue`) in a way that changes their ordering; the structure won't notice and will break.
- `null` keys are not allowed in `TreeMap` with natural ordering.

Need	Use
fast lookup, order irrelevant	<code>HashMap</code> / <code>HashSet</code> ($O(1)$ average)
keys in sorted order, floor/ceiling, ranges	<code>TreeMap</code> / <code>TreeSet</code> ($O(\log n)$)
insertion order	<code>LinkedHashMap</code>
repeatedly take the smallest/largest	<code>PriorityQueue</code>
sorted and thread-safe	<code>ConcurrentSkipListMap</code>

CHAPTER 15

Cheat Sheet: Choosing the Right Tree

15

15.1 Complexity of every tree

Structure	Search	Insert	Delete	Min / Max	Height	Notes
Binary tree (unordered)	$O(n)$	$O(1)^*$	$O(n)$	$O(n)$	any	*given the parent
BST (average)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	random order; average depth $\approx 1.39 \log_2 n$
BST (worst)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$n - 1$	sorted insertions
AVL tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\leq 1.44 \log n$	strictest balance
Red-black tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\leq 2 \log n$	fewest rotations; TreeMap
Splay tree	$O(\log n)$ am.	$O(\log n)$ am.	$O(\log n)$ am.	$O(\log n)$ am.	unbounded	recently used near the root
Treap	$O(\log n)$ exp.	$O(\log n)$ exp.	$O(\log n)$ exp.	$O(\log n)$ exp.	$O(\log n)$ exp.	random priorities
B-tree / B+ tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\log_t n$	minimal disk reads
Binary heap	$O(n)$	$O(\log n)$	$O(\log n)^{**}$	$O(1)$	$\log n$	**extract the top
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$	max word length	prefix queries in $O(L)$
Segment tree	query $O(\log n)$	update $O(\log n)$	—	—	$\log n$	any associative range op
Fenwick tree	query $O(\log n)$	update $O(\log n)$	—	—	$\log n$	prefix sums, tiny code
Union-find	find $O(\alpha(n))$	union $O(\alpha(n))$	—	—	tiny	connectivity

am. = amortized, exp. = expected.

15.2 Traversal quick reference

Order	Sequence	Use it for
Preorder	Node, Left, Right	copying, serializing, printing hierarchies
Inorder	Left, Node, Right	sorted output from a BST, validating a BST, k-th smallest
Postorder	Left, Right, Node	deleting, sizes/heights, evaluating expressions, folder sizes
Level order	level by level (queue)	levels, views, shortest distance from the root, heaps

15.3 Which tree should I use?

If you need...	Choose
a sorted map/set in memory	<code>TreeMap</code> / <code>TreeSet</code> (red-black)
the fastest lookups with rare updates	AVL tree (or a sorted array + binary search if static)
repeated "smallest/largest next"	binary heap (<code>PriorityQueue</code>)
an index on disk, range scans	B+ tree
prefix search, autocomplete, dictionaries	trie (or radix tree)
range sum/min/max with updates	segment tree (Fenwick for prefix sums)
grouping items into connected sets	union-find
optimal compression codes	Huffman tree
hierarchy with many children	N-ary tree
fast access to recently used keys	splay tree

15.4 The ten things to remember

1. A tree with n nodes has **$n - 1$ edges**; every node is the root of a subtree.
2. Tree operations cost **$O(\text{height})$** ; everything else is about keeping the height near $\log n$.
3. **Inorder of a BST is sorted.**
4. BST delete with two children: **replace with the inorder successor**.
5. Validate a BST with a **(low, high) window**.
6. **AVL**: $|b| \leq 1$; four cases LL, RR, LR, RL; single or double rotations.
7. **Red-black**: no red-red, equal black heights; height $\leq 2 \log n$; used by `TreeMap`.
8. **B-trees** are wide and shallow for disks; **B+ trees** keep data in linked leaves.
9. **Heaps** are complete trees in arrays: sift up, sift down, heapify in $O(n)$.
10. Most tree problems: **null base case, recurse left and right, combine** (return what the parent needs, record what the problem asks).